

THE LOVE FOR RECURSION

The Java stack, function calls and recursion.
A recurring dilemma.

Florian Sihler

September 21, 2026
SP, Ulm University

THE LOVE FOR RECURSION

The Java stack, function calls and recursion.
A recurring dilemma.

Florian Sihler



Officially supported by the Pingu-Foundation for Emotional Support. There will be light and there will be more peeps to come!

September 21, 2026
SP, Ulm University

2.1

Java's Memory Management

[5]: *The Java® Virtual Machine
Specification [Heap & Stack]*
Lindholm, 2020



The Java Stack



Function Calls



Theory



Examples



Wrap-Up



2.2

Java's Memory Management

```
int x = 0;  
byte y = 12;  
String f = "Hi_World";  
char[] c = {2, 3, 4};
```

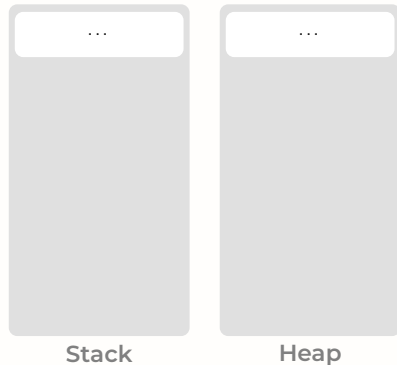
[5]: *The Java® Virtual Machine
Specification [Heap & Stack]*
Lindholm, 2020



2.3

Java's Memory Management

```
int x = 0;  
byte y = 12;  
String f = "Hi_World";  
char[] c = {2, 3, 4};
```



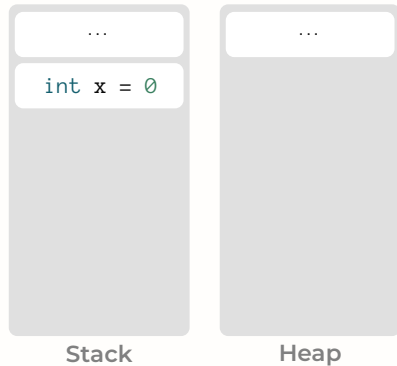
[5]: *The Java® Virtual Machine
Specification [Heap & Stack]*
Lindholm, 2020



2.4

Java's Memory Management

- ```
int x = 0;
byte y = 12;
String f = "Hi_World";
char[] c = {2, 3, 4};
```



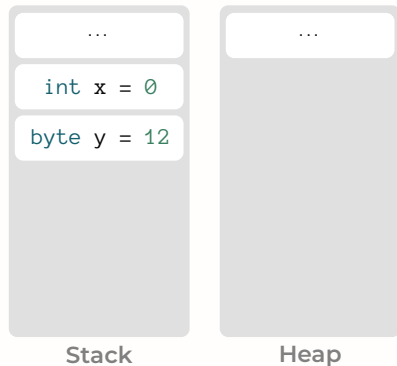
[5]: *The Java® Virtual Machine  
Specification [Heap & Stack]*  
Lindholm, 2020



## 2.5

# Java's Memory Management

```
int x = 0;
• byte y = 12;
String f = "Hi_World";
char[] c = {2, 3, 4};
```

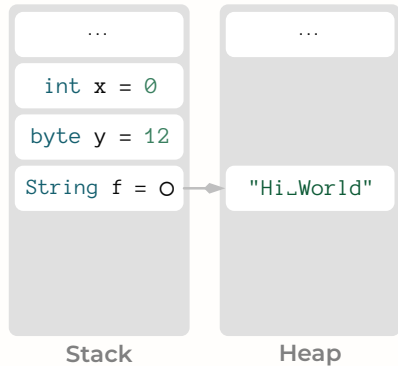


[5]: The Java® Virtual Machine  
Specification [Heap & Stack]  
Lindholm, 2020

## 2.6

# Java's Memory Management

- ```
int x = 0;
byte y = 12;
String f = "Hi_World";
char[] c = {2, 3, 4};
```

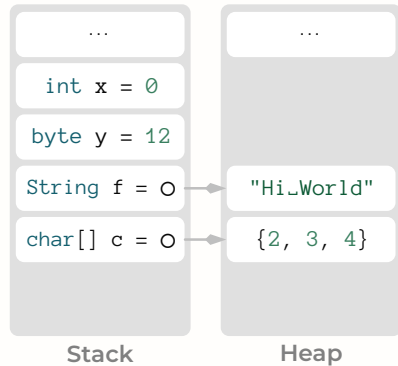


[5]: The Java® Virtual Machine
Specification [Heap & Stack]
Lindholm, 2020

2.7

Java's Memory Management

```
int x = 0;  
byte y = 12;  
String f = "Hi_World";  
• char[] c = {2, 3, 4};
```



3.1

More Complex Constructs

[5]: *The Java® Virtual Machine
Specification [Heap & Stack]*
Lindholm, 2020

[8]: *EPK-Package, tikzpingus*
Sihler, 2021

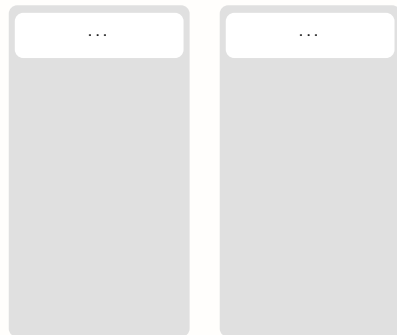


3.2

More Complex Constructs

```
class Mega {  
    public int i;  
    public String s;  
}
```

```
Mega mon = new Mega();  
mon.s = "Hello";  
mon.s = "World";
```



[5]: *The Java® Virtual Machine Specification [Heap & Stack]*
Lindholm, 2020

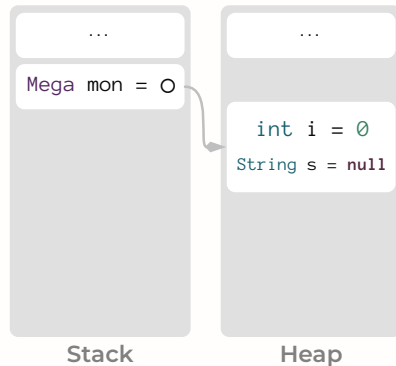
[8]: *ETK-Package, tikzpingus*
Sihler, 2021

3.3

More Complex Constructs

```
class Mega {  
    public int i;  
    public String s;  
}
```

- `Mega mon = new Mega();`
`mon.s = "Hello";`
`mon.s = "World";`



[5]: *The Java® Virtual Machine Specification [Heap & Stack]*
Lindholm, 2020

[8]: *ETK-Package, tikzpicture*
Sihler, 2021

Note: in Java every variable gets a “default value”. For complex data types this is **null**.

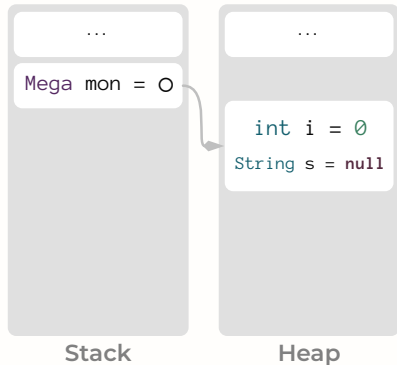


3.4

More Complex Constructs

```
class Mega {  
    public int i;  
    public String s;  
}
```

- `Mega mon = new Mega();`
`mon.s = "Hello";`
`mon.s = "World";`



[5]: The Java® Virtual Machine
Specification [Heap & Stack]
Lindholm, 2020

[8]: EPK-Package, tikzpingus
Sihler, 2021



Note: in Java every variable gets a “default value”. For complex data types this is **null**.



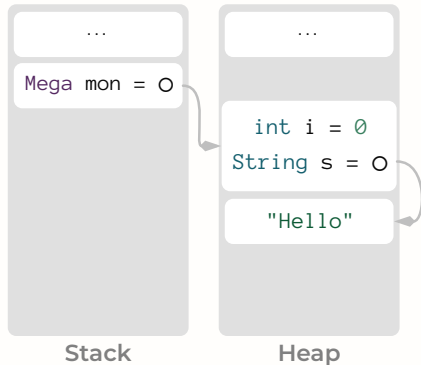
3.5

More Complex Constructs

```
class Mega {  
    public int i;  
    public String s;  
}
```

```
Mega mon = new Mega();
```

- `mon.s = "Hello";`
`mon.s = "World";`



[5]: *The Java® Virtual Machine Specification [Heap & Stack]*
Lindholm, 2020

[8]: *ETK-Package, tikzpingus*
Sihler, 2021



Note: in Java every variable gets a “default value”. For complex data types this is **null**.

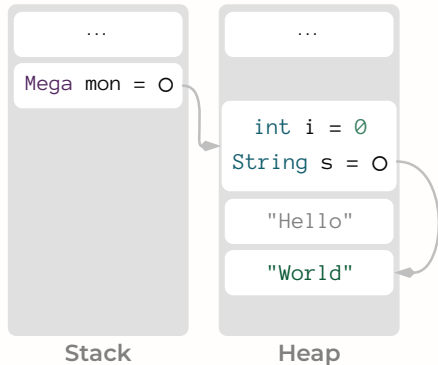


3.6

More Complex Constructs

```
class Mega {  
    public int i;  
    public String s;  
}
```

```
Mega mon = new Mega();  
mon.s = "Hello";  
• mon.s = "World";
```



[5]: The Java® Virtual Machine
Specification [Heap & Stack]
Lindholm, 2020

[8]: EPK-Package, tikzpingus
Sihler, 2021



4.1

Stack Frames

[3]: *Java Debug Interface –
StackFrame (JDK 11)*, 2021



The Java Stack



Function Calls



Theory



Examples



Wrap-Up



4.2

Stack Frames

- Handle (local) scopes.

4.3

Stack Frames

- Handle (local) scopes.
- Are pushed onto the stack when entered.

4.4

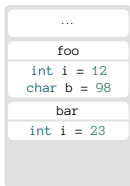
Stack Frames

- Handle (local) scopes.
- Are pushed onto the stack when entered.
- Are taken off the stack again when left.

4.5

Stack Frames

- Handle (local) scopes.
- Are pushed onto the stack when entered.
- Are taken off the stack again when left.



[3]: Java Debug Interface –
StackFrame (JDK 11)

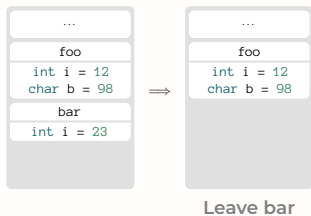
, 2021



4.6

Stack Frames

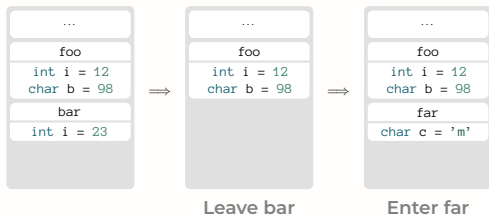
- Handle (local) scopes.
- Are pushed onto the stack when entered.
- Are taken off the stack again when left.



4.7

Stack Frames

- Handle (local) scopes.
- Are pushed onto the stack when entered.
- Are taken off the stack again when left.



[3]: Java Debug Interface –
StackFrame (JDK 11)

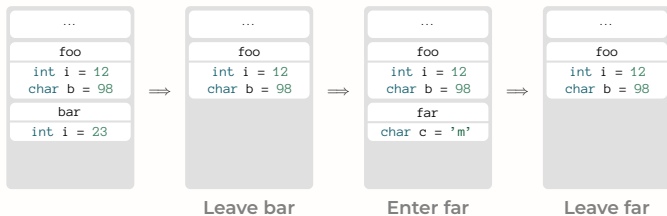
, 2021



4.8

Stack Frames

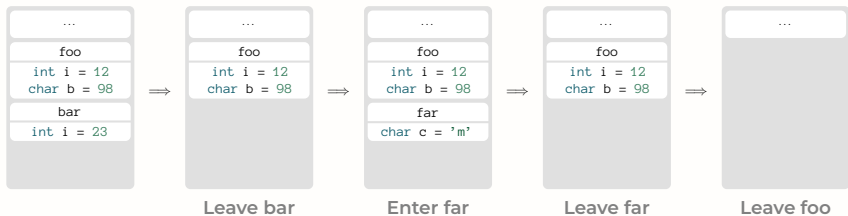
- Handle (local) scopes.
- Are pushed onto the stack when entered.
- Are taken off the stack again when left.



4.9

Stack Frames

- Handle (local) scopes.
- Are pushed onto the stack when entered.
- Are taken off the stack again when left.



[3]: Java Debug Interface –
StackFrame (JDK 11)

, 2021



5.1

Stack Frames – Example

[3]: *Java Debug Interface –
StackFrame (JDK 11)*, 2021



The Java Stack



Function Calls



Theory



Examples



Wrap-Up



5.2

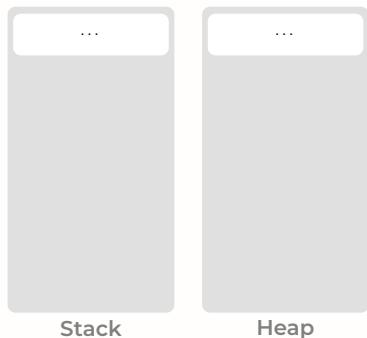
Stack Frames – Example

```
void f(char c) {  
    int x = 4;  
    System.out.print(b(c+x));  
}  
String b(int n) {  
    return "" + (n % 9);  
}  
  
f(b(42).charAt(0));
```

5.3

Stack Frames – Example

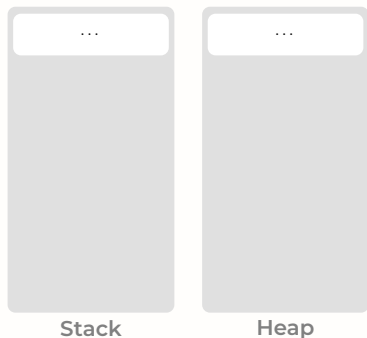
```
void f(char c) {  
    int x = 4;  
    System.out.print(b(c+x));  
}  
String b(int n) {  
    return "" + (n % 9);  
}  
  
f(b(42).charAt(0));
```



5.4

Stack Frames – Example

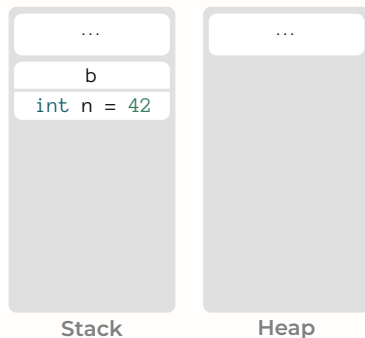
```
void f(char c) {  
    int x = 4;  
    System.out.print(b(c+x));  
}  
String b(int n) {  
    return "" + (n % 9);  
}  
  
f(b(42).charAt(0));
```



5.5

Stack Frames – Example

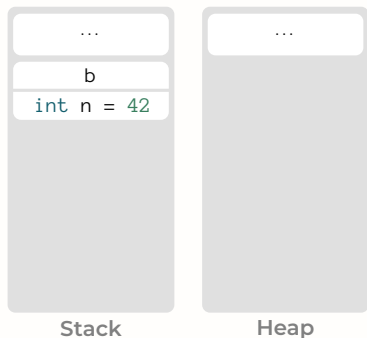
```
void f(char c) {  
    int x = 4;  
    System.out.print(b(c+x));  
}  
• String b(int n) {  
    return "" + (n % 9);  
}  
  
f(b(42).charAt(0));
```



5.6

Stack Frames – Example

```
void f(char c) {  
    int x = 4;  
    System.out.print(b(c+x));  
}  
String b(int n) {  
•   return "" + (n % 9);  
}  
  
f(b(42).charAt(0));
```

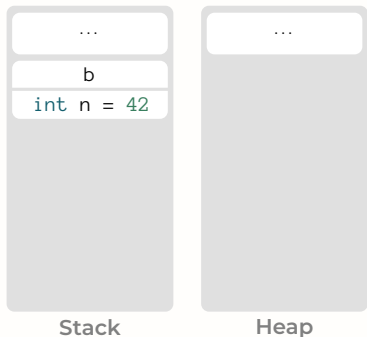




5.7

Stack Frames – Example

```
void f(char c) {  
    int x = 4;  
    System.out.print(b(c+x));  
}  
String b(int n) {  
•   return "" + (n % 9);  
}  
  
f(b(42).charAt(0));
```

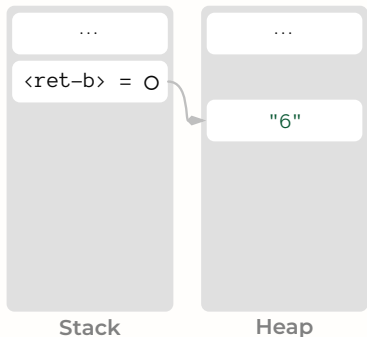




5.8

Stack Frames – Example

```
void f(char c) {  
    int x = 4;  
    System.out.print(b(c+x));  
}  
String b(int n) {  
    return "" + (n % 9);  
}  
  
f(b(42).charAt(0));
```

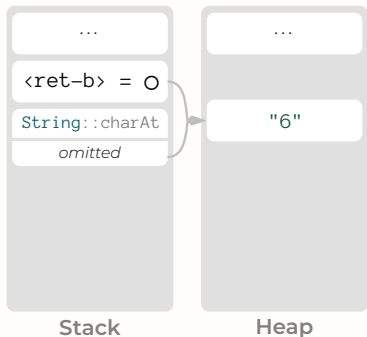




5.9

Stack Frames – Example

```
void f(char c) {  
    int x = 4;  
    System.out.print(b(c+x));  
}  
String b(int n) {  
    return "" + (n % 9);  
}  
  
f(b(42).charAt(0));
```



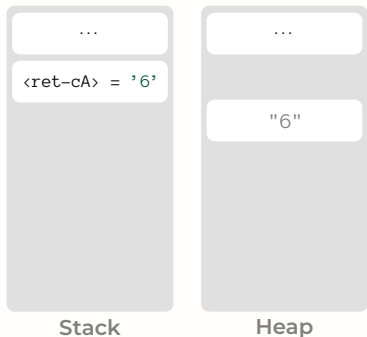


5.10

Stack Frames – Example

```
void f(char c) {  
    int x = 4;  
    System.out.print(b(c+x));  
}  
String b(int n) {  
    return "" + (n % 9);  
}
```

- `f(b(42).charAt(0));`



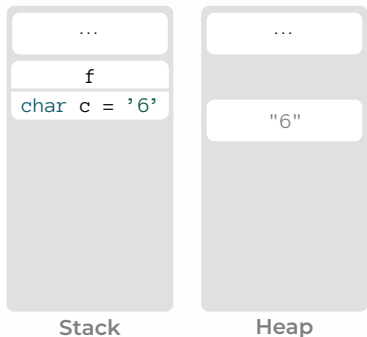


5.11

Stack Frames – Example

- ```
void f(char c) {
 int x = 4;
 System.out.print(b(c+x));
}
String b(int n) {
 return "" + (n % 9);
}

f(b(42).charAt(0));
```



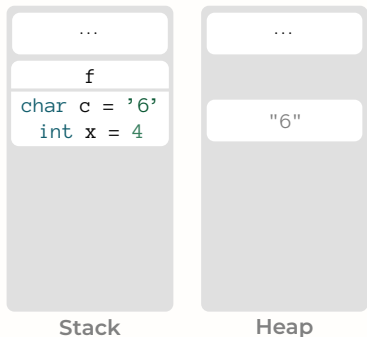


## 5.12

# Stack Frames – Example

```
void f(char c) {
 int x = 4;
 System.out.print(b(c+x));
}
String b(int n) {
 return "" + (n % 9);
}

f(b(42).charAt(0));
```



[3]: Java Debug Interface –  
StackFrame (JDK 11)

, 2021

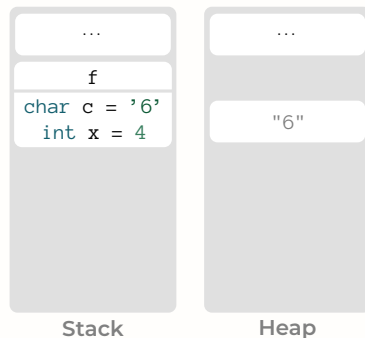


## 5.13

# Stack Frames – Example

```
void f(char c) {
 int x = 4;
 System.out.print(b(c+x));
}
String b(int n) {
 return "" + (n % 9);
}

f(b(42).charAt(0));
```



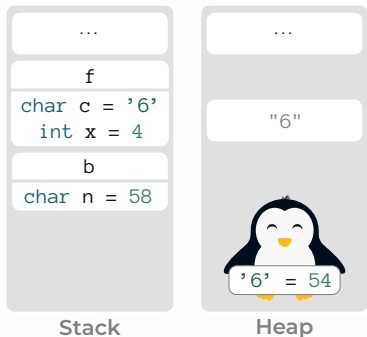


## 5.14

# Stack Frames – Example

```
void f(char c) {
 int x = 4;
 System.out.print(b(c+x));
}
• String b(int n) {
 return "" + (n % 9);
}

f(b(42).charAt(0));
```



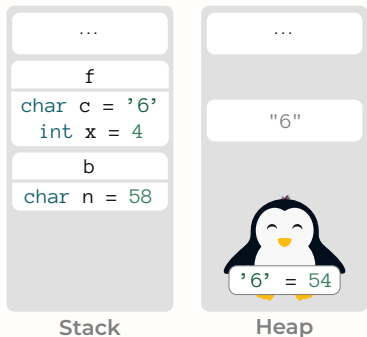


## 5.15

# Stack Frames – Example

```
void f(char c) {
 int x = 4;
 System.out.print(b(c+x));
}
String b(int n) {
• return "" + (n % 9);
}

f(b(42).charAt(0));
```



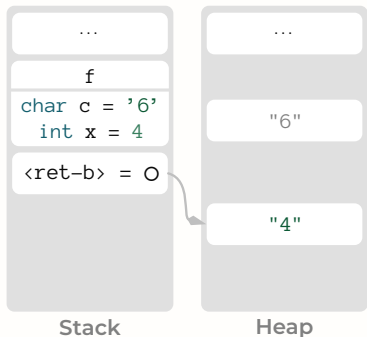


## 5.16

# Stack Frames – Example

```
void f(char c) {
 int x = 4;
 System.out.print(b(c+x));
}
String b(int n) {
 return "" + (n % 9);
}

f(b(42).charAt(0));
```



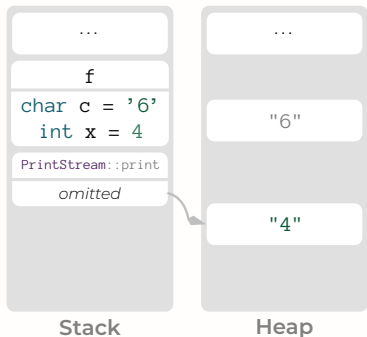




## 5.17

# Stack Frames – Example

- ```
void f(char c) {  
    int x = 4;  
    System.out.print(b(c+x));  
}  
String b(int n) {  
    return "" + (n % 9);  
}  
  
f(b(42).charAt(0));
```

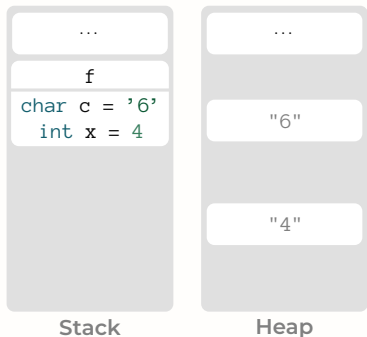




5.18

Stack Frames – Example

```
void f(char c) {  
    int x = 4;  
    System.out.print(b(c+x));  
}  
String b(int n) {  
    return "" + (n % 9);  
}  
  
f(b(42).charAt(0));
```



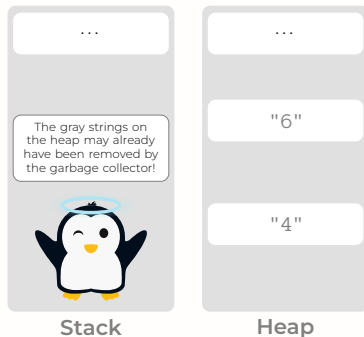
5.19

Stack Frames – Example

We are going to simplify return mechanisms shamefully!



```
void f(char c) {  
    int x = 4;  
    System.out.print(b(c+x));  
}  
String b(int n) {  
    return "" + (n % 9);  
}  
  
f(b(42).charAt(0)); •
```



[3]: Java Debug Interface –
StackFrame (JDK 11)

, 2021



6.1

Anatomy of a Recursion



6.2

Anatomy of a Recursion

```
1 func recursive(problem arguments) is
```

```
9 end
```



6.3

Anatomy of a Recursion

```
1 func recursive(problem arguments) is  
2   | if at elemental problem then  
   |  
4   | else  
   |  
8   | end  
9 end
```



6.4

Anatomy of a Recursion

```
1 func recursive(problem arguments) is  
2   | if at elemental problem then  
3   |   solve elemental problem;  
4   | else  
   |  
   |  
   |  
8   | end  
9 end
```



6.5

Anatomy of a Recursion

```
1 func recursive(problem arguments) is  
2   | if at elemental problem then  
3   |   solve elemental problem;  
4   | else  
5   |   Split problem into smaller ones;  
   |  
8   | end  
9 end
```



6.6

Anatomy of a Recursion

```
1 func recursive(problem arguments) is  
2   if at elemental problem then  
3     solve elemental problem;  
4   else  
5     Split problem into smaller ones;  
6     Try to solve them recursively;  
8   end  
9 end
```

6.7

Anatomy of a Recursion

```
1 func recursive(problem arguments) is  
2   if at elemental problem then  
3     solve elemental problem;  
4   else  
5     Split problem into smaller ones;  
6     Try to solve them recursively;  
7     Combine step-solutions to one;  
8   end  
9 end
```

6.8

Anatomy of a Recursion

```
1 func recursive(problem arguments) is  
2   if at elemental problem then  
3     solve elemental problem;  
4   else  
5     Split problem into smaller ones;  
6     Try to solve them recursively;  
7     Combine step-solutions to one;  
8   end  
9 end
```



Not all of them
have to follow ex-
actly this structure.
Be special. Be wild!

7.1

Stack Frames and Call Graphs



7.2

Stack Frames and Call Graphs

- Stack frames quickly get confusing.

7.3

Stack Frames and Call Graphs

- Stack frames quickly get confusing.
- We simplify this with call graphs.

7.4

Stack Frames and Call Graphs

- Stack frames quickly get confusing.
- We simplify this with call graphs.
- Nodes represent stack frames.

7.5

Stack Frames and Call Graphs

- Stack frames quickly get confusing.
- We simplify this with call graphs.
- Nodes represent stack frames.

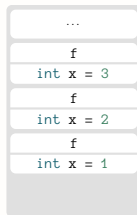
```
int f(int x) {  
    return f(x - 1);  
}
```


7.6

Stack Frames and Call Graphs

- Stack frames quickly get confusing.
- We simplify this with call graphs.
- Nodes represent stack frames.

```
int f(int x) {  
    return f(x - 1);  
}
```

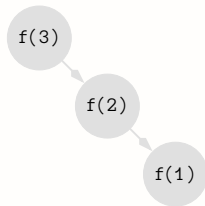
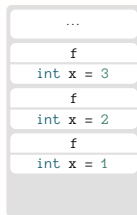


7.7

Stack Frames and Call Graphs

- Stack frames quickly get confusing.
- We simplify this with call graphs.
- Nodes represent stack frames.

```
int f(int x) {  
    return f(x - 1);  
}
```



8.1

Linear Recursion

[2]: *About primitive recursive algorithms*
Colson, 1991



The Java Stack



Function Calls



Theory



Examples



Wrap-Up



8.2

Linear Recursion

- ```
int f(int n) {
 if (n <= 0) return n;
 else return n + f(n - 1);
}
```

## 8.3

# Linear Recursion

```
int f(int n) {
 if (n <= 0) return n;
 • else return n + f(n - 1);
}
```

f(3)

## 8.4

# Linear Recursion

```
int f(int n) {
 if (n <= 0) return n;
 else return n + f(n - 1);
}
```



[2]: *About primitive recursive algorithms*  
Colson, 1991

## 8.5

# Linear Recursion

```
int f(int n) {
 if (n <= 0) return n;
 else return n + f(n - 1);
}
```

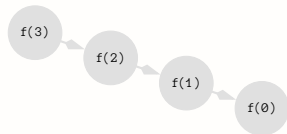


[2]: *About primitive recursive algorithms*  
Colson, 1991

## 8.6

# Linear Recursion

```
int f(int n) {
• if (n <= 0) return n;
 else return n + f(n - 1);
}
```



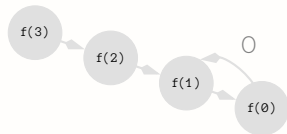
[2]: *About primitive recursive algorithms*  
Colson, 1991



## 8.7

# Linear Recursion

```
int f(int n) {
 if (n <= 0) return n;
 else return n + f(n - 1);
}
```

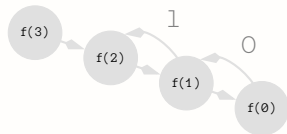


[2]: *About primitive recursive algorithms*  
Colson, 1991

## 8.8

# Linear Recursion

```
int f(int n) {
 if (n <= 0) return n;
 else return n + f(n - 1);
}
```

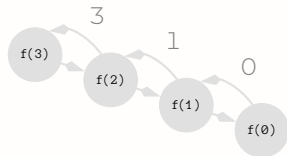


[2]: About primitive recursive algorithms  
Colson, 1991

## 8.9

# Linear Recursion

```
int f(int n) {
 if (n <= 0) return n;
 else return n + f(n - 1);
}
```

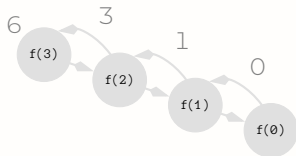


[2]: *About primitive recursive algorithms*  
Colson, 1991

## 8.10

# Linear Recursion

```
int f(int n) {
 if (n <= 0) return n;
 else return n + f(n - 1);
}
```

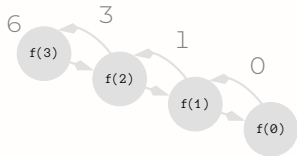


[2]: *About primitive recursive algorithms*  
Colson, 1991

## 8.11

# Linear Recursion

```
int f(int n) {
 if (n <= 0) return n;
 else return n + f(n - 1);
}
```



The simplest and most common form of recursion.

This one is even a *primitive recursive* function!<sup>[2]</sup>

[2]: About primitive recursive algorithms  
Colson, 1991

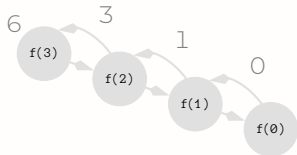
## 8.12

## Linear Recursion

Linear recursion is subdivided further! For example into head and tail recursion.



```
int f(int n) {
 if (n <= 0) return n;
 else return n + f(n - 1);
}
```



The simplest and most common form of recursion.

This one is even a *primitive recursive* function!<sup>[2]</sup>

[2]: About primitive recursive algorithms  
Colson, 1991



## 9.1

# Nested Recursion

[9]: *The Ackermann function, a theoretical, computational, and formula manipulative study*  
Sundblad, 1971



The Java Stack



Function Calls



**Theory**



Examples



Wrap-Up



## 9.2

# Nested Recursion

This is the Ackermann function



```
int a(int m, int n) {
 if(m == 0) return n + 1;
 else if (n == 0) return a(m - 1, 1);
 else return a(m - 1, a(m, n - 1));
}
```

[9]: *The Ackermann function, a theoretical, computational, and formula manipulative study*  
Sundblad, 1971



The Java Stack



Function Calls



Theory



Examples



Wrap-Up





## 9.3

# Nested Recursion

This is the Ackermann function



`a(1, 1)`

```
int a(int m, int n) {
 if(m == 0) return n + 1;
 else if (n == 0) return a(m - 1, 1);
 • else return a(m - 1, a(m, n - 1));
}
```

[9]: The Ackermann function, a theoretical, computational, and formula manipulative study  
Sundblad, 1971



## 9.4

# Nested Recursion

This is the Ackermann function



```
int a(int m, int n) {
 if(m == 0) return n + 1;
 else if (n == 0) return a(m - 1, 1);
 else return a(m - 1, a(m, n - 1));
}
```

a(1, 1)



a(1, 0)

[9]: The Ackermann function, a theoretical, computational, and formula manipulative study  
Sundblad, 1971



## 9.5

# Nested Recursion

This is the Ackermann function



```
int a(int m, int n) {
 if(m == 0) return n + 1;
 • else if (n == 0) return a(m - 1, 1);
 else return a(m - 1, a(m, n - 1));
}
```

a(1, 1)



a(1, 0)

[9]: The Ackermann function, a theoretical, computational, and formula manipulative study  
Sundblad, 1971

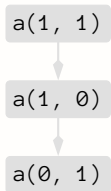
## 9.6

# Nested Recursion

This is the Ackermann function



```
int a(int m, int n) {
• if(m == 0) return n + 1;
 else if (n == 0) return a(m - 1, 1);
 else return a(m - 1, a(m, n - 1));
}
```



[9]: The Ackermann function, a theoretical, computational, and formula manipulative study  
Sundblad, 1971

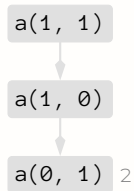
## 9.7

# Nested Recursion

This is the Ackermann function



```
int a(int m, int n) {
• if(m == 0) return n + 1;
 else if (n == 0) return a(m - 1, 1);
 else return a(m - 1, a(m, n - 1));
}
```



[9]: The Ackermann function, a theoretical, computational, and formula manipulative study  
Sundblad, 1971

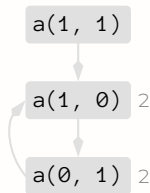
## 9.8

# Nested Recursion

This is the Ackermann function



```
int a(int m, int n) {
 if(m == 0) return n + 1;
 else if (n == 0) return a(m - 1, 1);
 else return a(m - 1, a(m, n - 1));
}
```



[9]: The Ackermann function, a theoretical, computational, and formula manipulative study  
Sundblad, 1971

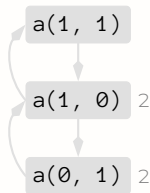
## 9.9

# Nested Recursion

This is the Ackermann function



```
int a(int m, int n) {
 if(m == 0) return n + 1;
 else if (n == 0) return a(m - 1, 1);
 else return a(m - 1, a(m, n - 1));
}
```



[9]: The Ackermann function, a theoretical, computational, and formula manipulative study  
Sundblad, 1971



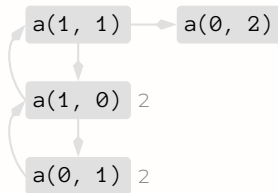
## 9.10

# Nested Recursion

This is the Ackermann function



```
int a(int m, int n) {
 if(m == 0) return n + 1;
 else if (n == 0) return a(m - 1, 1);
 else return a(m - 1, a(m, n - 1));
}
```



[9]: The Ackermann function, a theoretical, computational, and formula manipulative study  
Sundblad, 1971



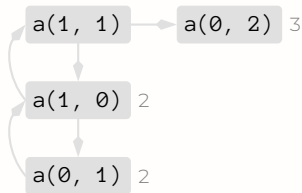
## 9.11

## Nested Recursion

This is the Ackermann function



```
int a(int m, int n) {
 if(m == 0) return n + 1;
 else if (n == 0) return a(m - 1, 1);
 else return a(m - 1, a(m, n - 1));
}
```



[9]: The Ackermann function, a theoretical, computational, and formula manipulative study  
Sundblad, 1971

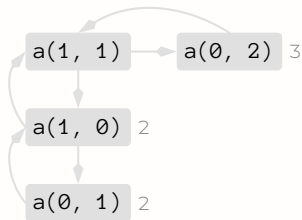
## 9.12

## Nested Recursion

This is the Ackermann function



```
int a(int m, int n) {
 if(m == 0) return n + 1;
 else if (n == 0) return a(m - 1, 1);
 else return a(m - 1, a(m, n - 1));
}
```



[9]: The Ackermann function, a theoretical, computational, and formula manipulative study  
Sundblad, 1971

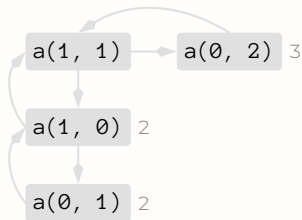
## 9.13

## Nested Recursion

This is the Ackermann function



```
int a(int m, int n) {
 if(m == 0) return n + 1;
 else if (n == 0) return a(m - 1, 1);
 else return a(m - 1, a(m, n - 1));
}
```

Hence  $a(1, 1) = 3$ .

[9]: The Ackermann function, a theoretical, computational, and formula manipulative study  
Sundblad, 1971

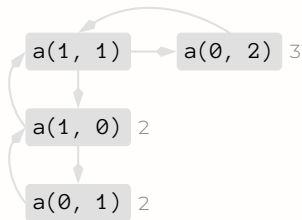
## 9.14

## Nested Recursion

This is the Ackermann function



```
int a(int m, int n) {
 if(m == 0) return n + 1;
 else if (n == 0) return a(m - 1, 1);
 else return a(m - 1, a(m, n - 1));
}
```

Hence  $a(1, 1) = 3$ .So what is, for example,  $a(4, 2)$ ?

[9]: The Ackermann function, a theoretical, computational, and formula manipulative study  
Sundblad, 1971

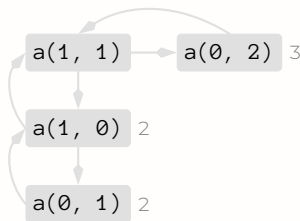
## 9.15

## Nested Recursion

This is the Ackermann function



```
int a(int m, int n) {
 if(m == 0) return n + 1;
 else if (n == 0) return a(m - 1, 1);
 else return a(m - 1, a(m, n - 1));
}
```

Hence  $a(1, 1) = 3$ .So what is, for example,  $a(4, 2)$ ?Well...  $a(4, 2) \approx 2 \cdot 10^{19729}$ .

[9]: The Ackermann function, a theoretical, computational, and formula manipulative study  
Sundblad, 1971



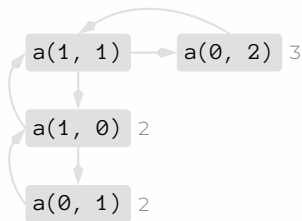
## 9.16

## Nested Recursion

This is the Ackermann function



```
int a(int m, int n) {
 if(m == 0) return n + 1;
 else if (n == 0) return a(m - 1, 1);
 else return a(m - 1, a(m, n - 1));
}
```



Nested recursive functions  
are usually very hard to follow.



Hence  $a(1, 1) = 3$ .

So what is, for example,  $a(4, 2)$ ?

Well...  $a(4, 2) \approx 2 \cdot 10^{19729}$ .

[9]: The Ackermann function, a  
theoretical, computational, and  
formula manipulative study  
Sundblad, 1971

## 10.1

# Cascading Recursion

[7]: *Exploring Recursion with  
Fibonacci Numbers*  
Rubio-Sánchez, 2007



The Java Stack



Function Calls



**Theory**



Examples



Wrap-Up



## 10.2

# Cascading Recursion

```
int f(int n) {
 if(n <= 2) return 1;
 else return f(n - 1) + f(n - 2);
}
```

[7]: Exploring Recursion with  
Fibonacci Numbers  
Rubio-Sánchez, 2007





## 10.3

# Cascading Recursion

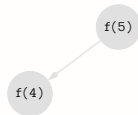
f(5)

- ```
int f(int n) {  
    if(n <= 2) return 1;  
    else return f(n - 1) + f(n - 2);  
}
```

10.4

Cascading Recursion

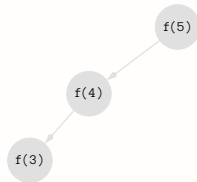
```
int f(int n) {  
    if(n <= 2) return 1;  
    else return f(n - 1) + f(n - 2);  
}
```



10.5

Cascading Recursion

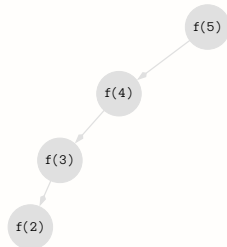
```
int f(int n) {  
    if(n <= 2) return 1;  
    else return f(n - 1) + f(n - 2);  
}
```



10.6

Cascading Recursion

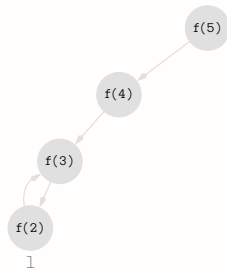
```
int f(int n) {  
    if(n <= 2) return 1;  
    else return f(n - 1) + f(n - 2);  
}
```



10.7

Cascading Recursion

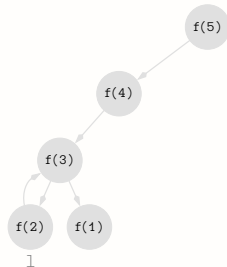
- ```
int f(int n) {
 if(n <= 2) return 1;
 else return f(n - 1) + f(n - 2);
}
```



## 10.8

# Cascading Recursion

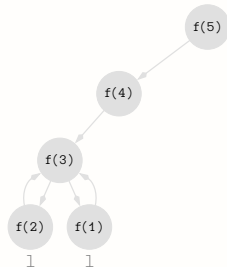
```
int f(int n) {
 if(n <= 2) return 1;
 else return f(n - 1) + f(n - 2);
}
```



## 10.9

# Cascading Recursion

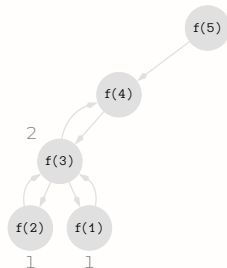
- ```
int f(int n) {  
    if(n <= 2) return 1;  
    else return f(n - 1) + f(n - 2);  
}
```



10.10

Cascading Recursion

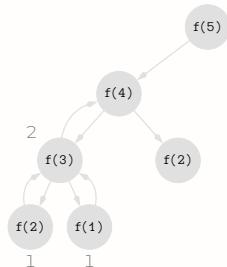
```
int f(int n) {  
    if(n <= 2) return 1;  
    else return f(n - 1) + f(n - 2);  
}
```



10.11

Cascading Recursion

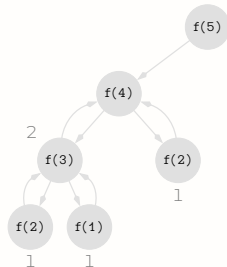
```
int f(int n) {  
    if(n <= 2) return 1;  
    else return f(n - 1) + f(n - 2);  
}
```



10.12

Cascading Recursion

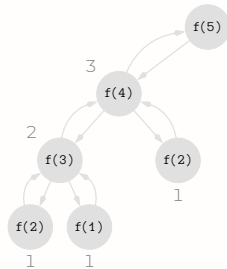
- ```
int f(int n) {
 if(n <= 2) return 1;
 else return f(n - 1) + f(n - 2);
}
```



## 10.13

# Cascading Recursion

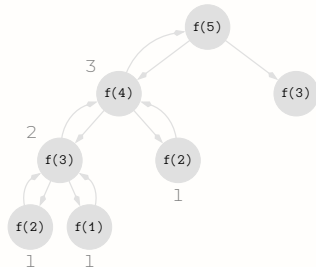
```
int f(int n) {
 if(n <= 2) return 1;
 else return f(n - 1) + f(n - 2);
}
```



## 10.14

# Cascading Recursion

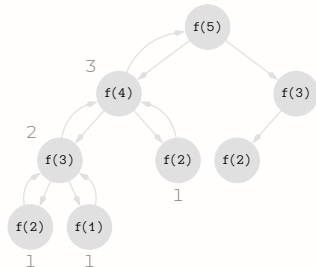
```
int f(int n) {
 if(n <= 2) return 1;
 else return f(n - 1) + f(n - 2);
}
```



## 10.15

# Cascading Recursion

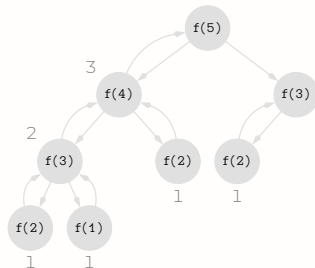
```
int f(int n) {
 if(n <= 2) return 1;
 else return f(n - 1) + f(n - 2);
}
```



## 10.16

# Cascading Recursion

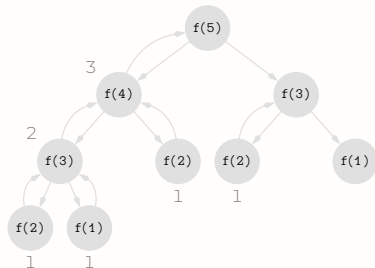
```
int f(int n) {
 if(n <= 2) return 1;
 else return f(n - 1) + f(n - 2);
}
```



## 10.17

# Cascading Recursion

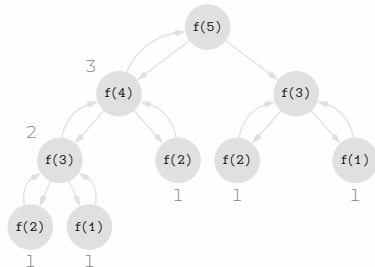
```
int f(int n) {
 if(n <= 2) return 1;
 else return f(n - 1) + f(n - 2);
}
```



## 10.18

# Cascading Recursion

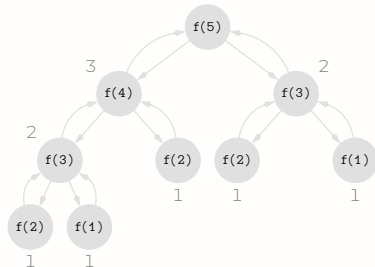
- ```
int f(int n) {  
    if(n <= 2) return 1;  
    else return f(n - 1) + f(n - 2);  
}
```



10.19

Cascading Recursion

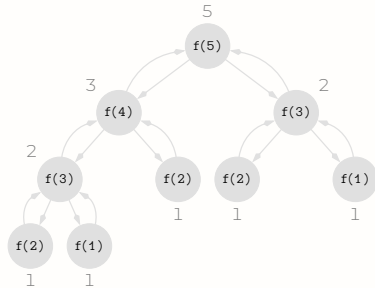
```
int f(int n) {  
    if(n <= 2) return 1;  
    else return f(n - 1) + f(n - 2);  
}
```



10.20

Cascading Recursion

```
int f(int n) {  
    if(n <= 2) return 1;  
    else return f(n - 1) + f(n - 2);  
}
```

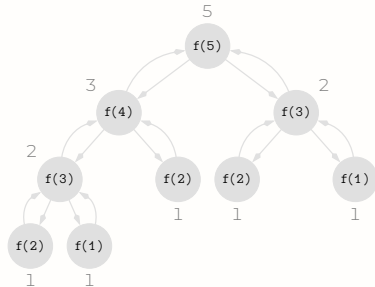


10.21

Cascading Recursion

```
int f(int n) {  
    if(n <= 2) return 1;  
    else return f(n - 1) + f(n - 2);  
}
```

Elegant, but usually not particularly efficient.



It is sometimes also called
"branching recursion".

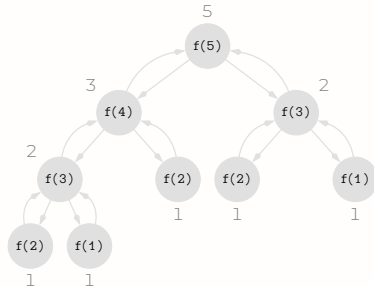


10.22

Cascading Recursion

```
int f(int n) {  
    if(n <= 2) return 1;  
    else return f(n - 1) + f(n - 2);  
}
```

Elegant, but usually not
particularly efficient.



[7]: Exploring Recursion with
Fibonacci Numbers
Rubio-Sánchez, 2007



11.1

Mutual Recursion



11.2

Mutual Recursion

```
int a(int x) {  
    if(x <= 0) return x + 1;  
    return b(x - 1);  
}  
  
int b(int x) {  
    if(x <= 0) return x;  
    return a(x % 3);  
}
```



11.3

Mutual Recursion

- ```
int a(int x) {
 if(x <= 0) return x + 1;
 return b(x - 1);
}

int b(int x) {
 if(x <= 0) return x;
 return a(x % 3);
}
```

a(42)

With function tuples it can  
always be transformed  
into “normal” recursion.



## 11.4

# Mutual Recursion

```
int a(int x) {
 if(x <= 0) return x + 1;
 return b(x - 1);
}

int b(int x) {
 if(x <= 0) return x;
 return a(x % 3);
}
```





With function tuples it can  
always be transformed  
into “normal” recursion.



## 11.5

# Mutual Recursion

```
int a(int x) {
 if(x <= 0) return x + 1;
 return b(x - 1);
}
```

```
int b(int x) {
 if(x <= 0) return x;
 • return a(x % 3);
}
```



With function tuples it can  
always be transformed  
into “normal” recursion.

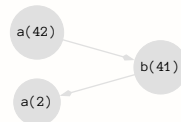


## 11.6

# Mutual Recursion

```
int a(int x) {
 if(x <= 0) return x + 1;
 return b(x - 1);
}

int b(int x) {
 if(x <= 0) return x;
 return a(x % 3);
}
```



With function tuples it can  
always be transformed  
into “normal” recursion.

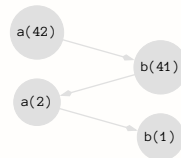


## 11.7

# Mutual Recursion

```
int a(int x) {
 if(x <= 0) return x + 1;
 return b(x - 1);
}

int b(int x) {
 if(x <= 0) return x;
 return a(x % 3);
}
```



With function tuples it can  
always be transformed  
into “normal” recursion.

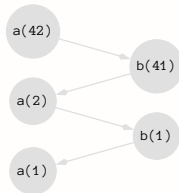


## 11.8

# Mutual Recursion

```
int a(int x) {
 if(x <= 0) return x + 1;
 return b(x - 1);
}

int b(int x) {
 if(x <= 0) return x;
 return a(x % 3);
}
```



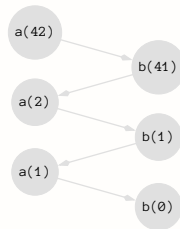


## 11.9

# Mutual Recursion

```
int a(int x) {
 if(x <= 0) return x + 1;
 return b(x - 1);
}

int b(int x) {
 if(x <= 0) return x;
 return a(x % 3);
}
```



With function tuples it can  
always be transformed  
into “normal” recursion.

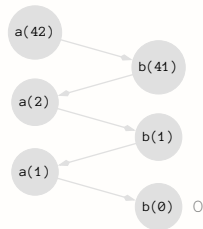


## 11.10

# Mutual Recursion

```
int a(int x) {
 if(x <= 0) return x + 1;
 return b(x - 1);
}

int b(int x) {
 if(x <= 0) return x;
 return a(x % 3);
}
```



With function tuples it can  
always be transformed  
into “normal” recursion.

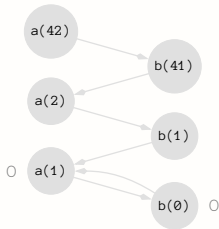


## 11.11

# Mutual Recursion

```
int a(int x) {
 if(x <= 0) return x + 1;
 return b(x - 1);
}

int b(int x) {
 if(x <= 0) return x;
 return a(x % 3);
}
```



With function tuples it can  
always be transformed  
into “normal” recursion.

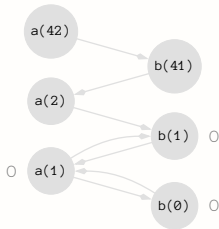


## 11.12

# Mutual Recursion

```
int a(int x) {
 if(x <= 0) return x + 1;
 return b(x - 1);
}

int b(int x) {
 if(x <= 0) return x;
 return a(x % 3);
}
```





With function tuples it can  
always be transformed  
into “normal” recursion.

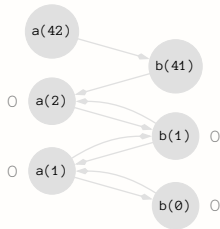


## 11.13

# Mutual Recursion

```
int a(int x) {
 if(x <= 0) return x + 1;
 return b(x - 1);
}

int b(int x) {
 if(x <= 0) return x;
 return a(x % 3);
}
```



With function tuples it can  
always be transformed  
into “normal” recursion.

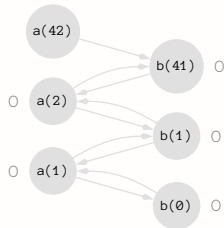


## 11.14

# Mutual Recursion

```
int a(int x) {
 if(x <= 0) return x + 1;
 return b(x - 1);
}

int b(int x) {
 if(x <= 0) return x;
 return a(x % 3);
}
```



With function tuples it can  
always be transformed  
into “normal” recursion.

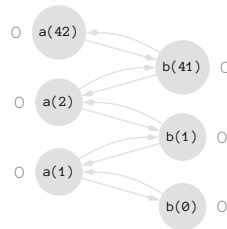


## 11.15

# Mutual Recursion

```
int a(int x) {
 if(x <= 0) return x + 1;
 return b(x - 1);
}

int b(int x) {
 if(x <= 0) return x;
 return a(x % 3);
}
```



## 12.1

# A Few Notes

[6]: *From Recursion to Iteration:  
What Are the Optimizations?*  
Liu, 1999



The Java Stack



Function Calls



**Theory**



Examples



Wrap-Up



## 12.2

# A Few Notes

- The examples shown were kept as minimal as possible

[6]: *From Recursion to Iteration:  
What Are the Optimizations?*  
Liu, 1999



The Java Stack



Function Calls



**Theory**



Examples



Wrap-Up



## 12.3

### A Few Notes

- The examples shown were kept as minimal as possible
- The variants can be combined!

[6]: *From Recursion to Iteration:  
What Are the Optimizations?*  
Liu, 1999



The Java Stack



Function Calls



**Theory**



Examples



Wrap-Up



## 12.4

### A Few Notes

- The examples shown were kept as minimal as possible
- The variants can be combined!
  - › e.g. a cascading, mutual recursion.

## 12.5

### A Few Notes

- The examples shown were kept as minimal as possible
- The variants can be combined!
  - › e.g. a cascading, mutual recursion.
- Recursion and iteration are usually equally powerful.



## 13.1

# Head vs. Tail Recursion



## 13.2

## Head vs. Tail Recursion

```
void h(int n) {
 if(n <= 0) return;
 h(n - 1);
 System.out.println(n);
}
```

```
void t(int n) {
 if(n <= 0) return;
 System.out.println(n);
 t(n - 1);
}
```



## 13.3

# Head vs. Tail Recursion

- ```
void h(int n) {  
    if(n <= 0) return;  
    h(n - 1);  
    System.out.println(n);  
}
```

h(3)

- ```
void t(int n) {
 if(n <= 0) return;
 System.out.println(n);
 t(n - 1);
}
```

t(3)

## 13.4

# Head vs. Tail Recursion

```
void h(int n) {
 if(n <= 0) return;
 • h(n - 1);
 System.out.println(n);
}
```

h(3)

```
void t(int n) {
 if(n <= 0) return;
 • System.out.println(n);
 t(n - 1);
}
```

t(3)

## 13.5

# Head vs. Tail Recursion

```
void h(int n) {
 if(n <= 0) return;
 • h(n - 1);
 System.out.println(n);
}
```



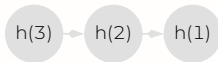
```
void t(int n) {
 if(n <= 0) return;
 • System.out.println(n);
 t(n - 1);
}
```



## 13.6

# Head vs. Tail Recursion

```
void h(int n) {
 if(n <= 0) return;
 • h(n - 1);
 System.out.println(n);
}
```



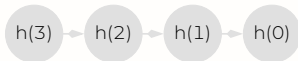
```
void t(int n) {
 if(n <= 0) return;
 System.out.println(n);
 • t(n - 1);
}
```



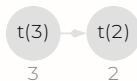
## 13.7

# Head vs. Tail Recursion

```
void h(int n) {
 • if(n <= 0) return;
 h(n - 1);
 System.out.println(n);
}
```



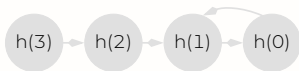
```
void t(int n) {
 • if(n <= 0) return;
 System.out.println(n);
 t(n - 1);
}
```



## 13.8

# Head vs. Tail Recursion

```
void h(int n) {
 if(n <= 0) return;
 h(n - 1);
 System.out.println(n);
}
```



```
void t(int n) {
 if(n <= 0) return;
 System.out.println(n);
 t(n - 1);
}
```

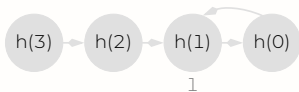




## 13.9

# Head vs. Tail Recursion

```
void h(int n) {
 if(n <= 0) return;
 h(n - 1);
 System.out.println(n);
}
```



```
void t(int n) {
 if(n <= 0) return;
 System.out.println(n);
 t(n - 1);
}
```

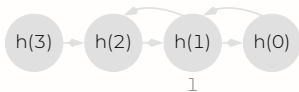


## 13.10

# Head vs. Tail Recursion

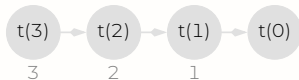
```
void h(int n) {
 if(n <= 0) return;
 h(n - 1);
 System.out.println(n);
}
```

- 



```
void t(int n) {
 if(n <= 0) return;
 System.out.println(n);
 t(n - 1);
}
```

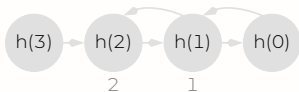
- 



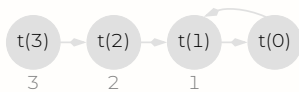
## 13.11

# Head vs. Tail Recursion

```
void h(int n) {
 if(n <= 0) return;
 h(n - 1);
 System.out.println(n);
}
```



```
void t(int n) {
 if(n <= 0) return;
 System.out.println(n);
 t(n - 1);
}
```

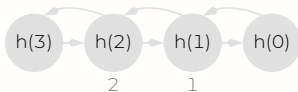


## 13.12

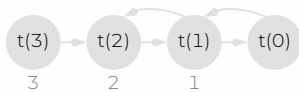
# Head vs. Tail Recursion

```
void h(int n) {
 if(n <= 0) return;
 h(n - 1);
 System.out.println(n);
}
```

- 



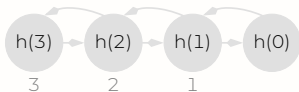
```
void t(int n) {
 if(n <= 0) return;
 System.out.println(n);
 t(n - 1);
}
```



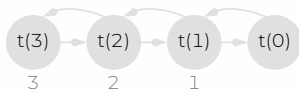
## 13.13

# Head vs. Tail Recursion

```
void h(int n) {
 if(n <= 0) return;
 h(n - 1);
 System.out.println(n);
}
```



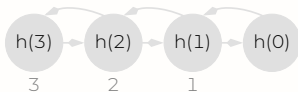
```
void t(int n) {
 if(n <= 0) return;
 System.out.println(n);
 t(n - 1);
}
```



## 13.14

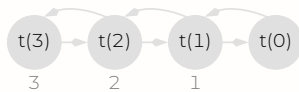
## Head vs. Tail Recursion

```
void h(int n) {
 if(n <= 0) return;
 h(n - 1);
 System.out.println(n);
}
```



Output: 1 → 2 → 3

```
void t(int n) {
 if(n <= 0) return;
 System.out.println(n);
 t(n - 1);
}
```



Output: 3 → 2 → 1

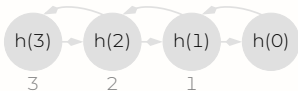
In short: does something still happen on the way back up the recursion or not (tail).



## 13.15

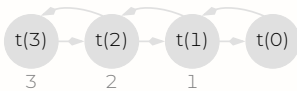
# Head vs. Tail Recursion

```
void h(int n) {
 if(n <= 0) return;
 h(n - 1);
 System.out.println(n);
}
```



Output: 1 → 2 → 3

```
void t(int n) {
 if(n <= 0) return;
 System.out.println(n);
 t(n - 1);
}
```



Output: 3 → 2 → 1



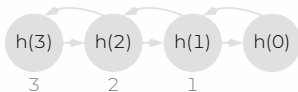
## 13.16

## Head vs. Tail Recursion

In short: does something still happen on the way back up the recursion or not (tail).

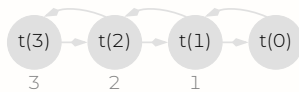


```
void h(int n) {
 if(n <= 0) return;
 h(n - 1);
 System.out.println(n);
}
```



Output: 1 → 2 → 3

```
void t(int n) {
 if(n <= 0) return;
 System.out.println(n);
 t(n - 1);
}
```



Output: 3 → 2 → 1

Disclaimer: the subtleties of head recursion are neglected here. Tail recursion is more interesting because it can be transformed into iteration without any trouble.





## 14.1

# Mathematical Functions



## 14.2

# Mathematical Functions

- Some mathematical functions are defined recursively.

## 14.3

# Mathematical Functions

- Some mathematical functions are defined recursively.
- The given definition can usually be translated 1:1.

## 14.4

# Mathematical Functions

- Some mathematical functions are defined recursively.
- The given definition can usually be translated 1:1.
- It already takes a good deal of the thinking off your hands.

## 14.5

# Mathematical Functions

- Some mathematical functions are defined recursively.
- The given definition can usually be translated 1:1.
- It already takes a good deal of the thinking off your hands.

$$f(n) = \begin{cases} 1, & n \leq 1 \\ n \cdot f(n-1), & n > 1 \end{cases}$$

$$f_n : \mathbb{N} \rightarrow \mathbb{N}, n \mapsto n \cdot f_{n-1}$$

$$f_1 = 1$$

## 14.6

# Mathematical Functions

- Some mathematical functions are defined recursively.
- The given definition can usually be translated 1:1.
- It already takes a good deal of the thinking off your hands.

$$f(n) = \begin{cases} 1, & n \leq 1 \\ n \cdot f(n-1), & n > 1 \end{cases}$$

$f_n : \mathbb{N} \rightarrow \mathbb{N}, n \mapsto n \cdot f_{n-1}$   
 $f_1 = 1$

```
public int f(int n) {

}
}
```

## 14.7

# Mathematical Functions

- Some mathematical functions are defined recursively.
- The given definition can usually be translated 1:1.
- It already takes a good deal of the thinking off your hands.

$$f(n) = \begin{cases} 1, & n \leq 1 \\ n \cdot f(n-1), & n > 1 \end{cases}$$

$$f_n : \mathbb{N} \rightarrow \mathbb{N}, n \mapsto n \cdot f_{n-1}$$

$$f_1 = 1$$

```
public int f(int n) {
 if(n <= 1) return 1;
}
```

## 14.8

# Mathematical Functions

- Some mathematical functions are defined recursively.
- The given definition can usually be translated 1:1.
- It already takes a good deal of the thinking off your hands.

$$f(n) = \begin{cases} 1, & n \leq 1 \\ n \cdot f(n-1), & n > 1 \end{cases}$$

$$f_n : \mathbb{N} \rightarrow \mathbb{N}, n \mapsto n \cdot f_{n-1}$$

$$f_1 = 1$$

```
public int f(int n) {
 if(n <= 1) return 1;
 else return n * f(n - 1);
}
```



## 14.9

# Mathematical Functions

- Some mathematical functions are defined recursively.
- The given definition can usually be translated 1:1.
- It already takes a good deal of the thinking off your hands.

"Elemental problems" are the non-recursive cases here: parts of the sequence that are often given axiomatically.



$$f(n) = \begin{cases} 1, & n \leq 1 \\ n \cdot f(n-1), & n > 1 \end{cases}$$

$$f_n : \mathbb{N} \rightarrow \mathbb{N}, n \mapsto n \cdot f_{n-1}$$

$$f_1 = 1$$

```
public int f(int n) {
 if(n <= 1) return 1;
 else return n * f(n - 1);
}
```

## 15.1

# The Ackermann Function, Alternative



## 15.2

# The Ackermann Function, Alternative

$$f(n) = \phi(n, n, n)$$



## 15.3

# The Ackermann Function, Alternative

$$f(n) = \phi(n, n, n)$$

$$\phi(a, b, 0) = a + b$$

$$\phi(a, 0, n) = \alpha(a, n - 1)$$

$$\phi(a, b, n) = \phi(a, \phi(a, b - 1, n), n - 1)$$



## 15.4

# The Ackermann Function, Alternative

$$f(n) = \phi(n, n, n)$$

$$\phi(a, b, 0) = a + b$$

$$\phi(a, 0, n) = \alpha(a, n - 1)$$

$$\phi(a, b, n) = \phi(a, \phi(a, b - 1, n), n - 1)$$

$$\alpha(a, n) = \begin{cases} 0, & \text{if } n = 0 \\ 1, & \text{if } n = 1 \\ a, & \text{if } n > 1 \end{cases}$$



## 15.5

## The Ackermann Function, Alternative



```
int f(int n) {
 return phi(n, n, n);
}

int alpha(int a, int n) {
 return n <= 1 ? n : a;
}
```



$$f(n) = \phi(n, n, n)$$

$$\phi(a, b, 0) = a + b$$

$$\phi(a, 0, n) = \alpha(a, n - 1)$$

$$\phi(a, b, n) = \phi(a, \phi(a, b - 1, n), n - 1)$$

$$\alpha(a, n) = \begin{cases} 0, & \text{if } n = 0 \\ 1, & \text{if } n = 1 \\ a, & \text{if } n > 1 \end{cases}$$



## 16.1

# The Ackermann Function, Alternative



## 16.2

## The Ackermann Function, Alternative



```
int phi(int a, int b, int n) {

}
```



## 16.3 The Ackermann Function, Alternative

## 11

```
int phi(int a, int b, int n) {

}
```

$$\phi(a, b, 0) = a + b$$

$$\phi(a, 0, n) = \alpha(a, n - 1)$$

$$\phi(a, b, n) = \phi(a, \phi(a, b - 1, n), n - 1)$$

## 16.4

# The Ackermann Function, Alternative



```
int phi(int a, int b, int n) {
 if(n == 0) return a + b;

}
```

$$\phi(a, b, 0) = a + b$$

$$\phi(a, 0, n) = \alpha(a, n - 1)$$

$$\phi(a, b, n) = \phi(a, \phi(a, b - 1, n), n - 1)$$



## 16.5

# The Ackermann Function, Alternative



```
int phi(int a, int b, int n) {
 if(n == 0) return a + b;
 else if (b == 0) return alpha(a, n - 1);
}
```

$$\phi(a, b, 0) = a + b$$

$$\phi(a, 0, n) = \alpha(a, n - 1)$$

$$\phi(a, b, n) = \phi(a, \phi(a, b - 1, n), n - 1)$$



## 16.6

## The Ackermann Function, Alternative



```
int phi(int a, int b, int n) {
 if(n == 0) return a + b;
 else if (b == 0) return alpha(a, n - 1);
 else return phi(a, phi(a, b - 1, n), n - 1);
}
```

$$\phi(a, b, 0) = a + b$$

$$\phi(a, 0, n) = \alpha(a, n - 1)$$

$$\phi(a, b, n) = \phi(a, \phi(a, b - 1, n), n - 1)$$



## 16.7

# The Ackermann Function, Alternative



```
int phi(int a, int b, int n) {
 if(n == 0) return a + b;
 else if (b == 0) return alpha(a, n - 1);
 else return phi(a, phi(a, b - 1, n), n - 1);
}
```

$f(1) = 1$

$$\phi(a, b, 0) = a + b$$

$$\phi(a, 0, n) = \alpha(a, n - 1)$$

$$\phi(a, b, n) = \phi(a, \phi(a, b - 1, n), n - 1)$$



## 16.8

# The Ackermann Function, Alternative



```
int phi(int a, int b, int n) {
 if(n == 0) return a + b;
 else if (b == 0) return alpha(a, n - 1);
 else return phi(a, phi(a, b - 1, n), n - 1);
}
```

$f(1) = 1, f(2) = 4$

$$\phi(a, b, 0) = a + b$$

$$\phi(a, 0, n) = \alpha(a, n - 1)$$

$$\phi(a, b, n) = \phi(a, \phi(a, b - 1, n), n - 1)$$



## 16.9

# The Ackermann Function, Alternative



```
int phi(int a, int b, int n) {
 if(n == 0) return a + b;
 else if (b == 0) return alpha(a, n - 1);
 else return phi(a, phi(a, b - 1, n), n - 1);
}
```

$f(1) = 1$ ,  $f(2) = 4$ ,  $f(3) = \text{⚡}$

$$\phi(a, b, 0) = a + b$$

$$\phi(a, 0, n) = \alpha(a, n - 1)$$

$$\phi(a, b, n) = \phi(a, \phi(a, b - 1, n), n - 1)$$



## 16.10

## The Ackermann Function, Alternative



```
int phi(int a, int b, int n) {
 if(n == 0) return a + b;
 else if (b == 0) return alpha(a, n - 1);
 else return phi(a, phi(a, b - 1, n), n - 1);
}
```

$f(1) = 1, f(2) = 4, f(3) = \text{⚡}$

$\gtrsim 2 \uparrow^2 7 \approx 10^{10^{10^{19000}}}$

$$\phi(a, b, 0) = a + b$$

$$\phi(a, 0, n) = \alpha(a, n - 1)$$

$$\phi(a, b, n) = \phi(a, \phi(a, b - 1, n), n - 1)$$





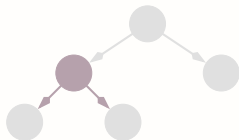
## 17.1

# Recursive Structures



## 17.2

# Recursive Structures

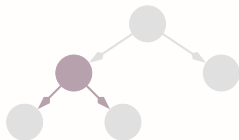


Every node is the root of a subtree



## 17.3

# Recursive Structures

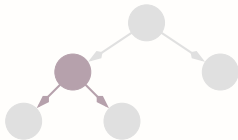


Every node is the root of a subtree

```
class Node {
 Node left;
 Node right;
 int data;
}
```

## 17.4

# Recursive Structures



Every node is the root of a subtree

```
class Node {
 Node left;
 Node right;
 int data;
}
```

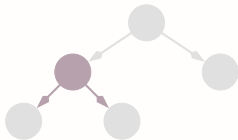
```
class Node<T> {
 Node left;
 Node right;
 T data;
}
```

Cooler with generics!<sup>[4]</sup>



## 17.5

# Recursive Structures



Every node is the root of a subtree

```
class Node {
 Node left;
 Node right;
 int data;
}
```

```
class Node<T> {
 Node left;
 Node right;
 T data;
}
```

Cooler with generics!<sup>[4]</sup>

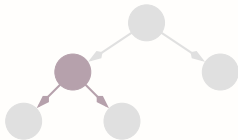


Every element is the head of a sublist



## 17.6

# Recursive Structures



Every node is the root of a subtree

```
class Node {
 Node left;
 Node right;
 int data;
}
```

```
class Node<T> {
 Node left;
 Node right;
 T data;
}
```

Cooler with generics!<sup>[4]</sup>



Every element is the head of a sublist

```
class Element {
 Element next;
 int data;
}
```



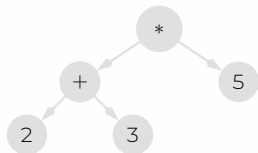
## 18.1

# Evaluating a Tree



## 18.2

# Evaluating a Tree

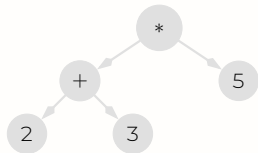


An arithmetic tree!



## 18.3

# Evaluating a Tree

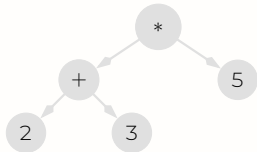


An arithmetic tree!

```
class Node {
 Node left, right;
 int data; String op;
}
```

## 18.4

# Evaluating a Tree



An arithmetic tree!

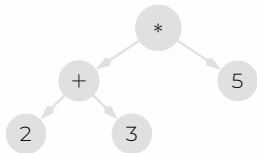
In practice one would realise the nodes differently. E.g. through inheritance.

```
class Node {
 Node left, right;
 int data; String op;
}
```



## 18.5

# Evaluating a Tree



An arithmetic tree!

```
long postOrderEval(Node node) {
```

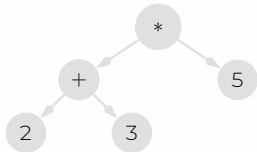
In practice one would realise the nodes differently. E.g. through inheritance.

```
class Node {
 Node left, right;
 int data; String op;
}
```



## 18.6

# Evaluating a Tree



An arithmetic tree!

```
long postOrderEval(Node node) {
 if(node == null) throw new /* ... */;
}
```

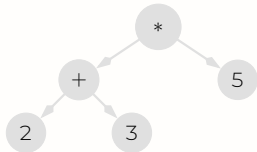
In practice one would realise the nodes differently. E.g. through inheritance.

```
class Node {
 Node left, right;
 int data; String op;
}
```



## 18.7

# Evaluating a Tree



An arithmetic tree!

```
long postOrderEval(Node node) {
 if(node == null) throw new /* ... */;
 if(node.op == null) // no operation
 return node.data;
}
```

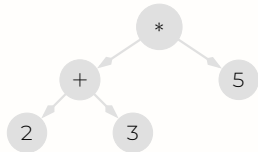
In practice one would re-  
alise the nodes differently.  
E.g. through inheritance.

```
class Node {
 Node left, right;
 int data; String op;
}
```



## 18.8

# Evaluating a Tree



An arithmetic tree!

In practice one would re-  
alise the nodes differently.  
E.g. through inheritance.

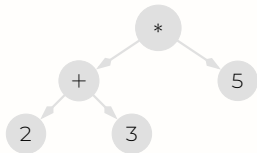


```
class Node {
 Node left, right;
 int data; String op;
}
```

```
long postOrderEval(Node node) {
 if(node == null) throw new /* ... */;
 if(node.op == null) // no operation
 return node.data;
 long l = postOrderEval(node.left);
 long r = postOrderEval(node.right);
```

## 18.9

# Evaluating a Tree



An arithmetic tree!

In practice one would realise the nodes differently. E.g. through inheritance.

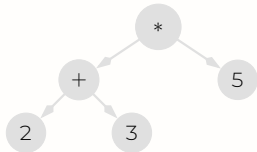
```
class Node {
 Node left, right;
 int data; String op;
}
```

```
long postOrderEval(Node node) {
 if(node == null) throw new /* ... */;
 if(node.op == null) // no operation
 return node.data;
 long l = postOrderEval(node.left);
 long r = postOrderEval(node.right);
 switch(node.op) {
 case "+":
 case "*":
 }
}
```



## 18.10

## Evaluating a Tree



An arithmetic tree!

```
class Node {
 Node left, right;
 int data; String op;
}
```

```
long postOrderEval(Node node) {
 if(node == null) throw new /* ... */;
 if(node.op == null) // no operation
 return node.data;
 long l = postOrderEval(node.left);
 long r = postOrderEval(node.right);
 switch(node.op) {
 case "+": return l + r;
 case "*":
 }
}
```

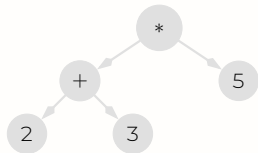
In practice one would realise the nodes differently. E.g. through inheritance.





## 18.11

# Evaluating a Tree



An arithmetic tree!

In practice one would realise the nodes differently. E.g. through inheritance.

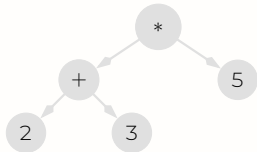
```
class Node {
 Node left, right;
 int data; String op;
}
```

```
long postOrderEval(Node node) {
 if(node == null) throw new /* ... */;
 if(node.op == null) // no operation
 return node.data;
 long l = postOrderEval(node.left);
 long r = postOrderEval(node.right);
 switch(node.op) {
 case "+": return l + r;
 case "*": return l * r;
 }
}
```



## 18.12

## Evaluating a Tree



An arithmetic tree!

```
class Node {
 Node left, right;
 int data; String op;
}
```

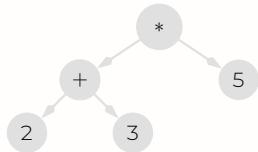
```
long postOrderEval(Node node) {
 if(node == null) throw new /* ... */;
 if(node.op == null) // no operation
 return node.data;
 long l = postOrderEval(node.left);
 long r = postOrderEval(node.right);
 switch(node.op) {
 case "+": return l + r;
 case "*": return l * r;
 // ... other cases, errors, ...
 }
}
```

In practice one would re-  
alise the nodes differently.  
E.g. through inheritance.



## 18.13

## Evaluating a Tree



An arithmetic tree!

In practice one would realise the nodes differently. E.g. through inheritance.

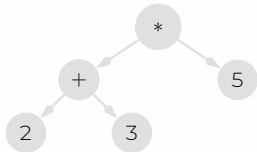
```
class Node {
 Node left, right;
 int data; String op;
}
```

```
long postOrderEval(Node node) {
 if(node == null) throw new /* ... */;
 if(node.op == null) // no operation
 return node.data;
 long l = postOrderEval(node.left);
 long r = postOrderEval(node.right);
 switch(node.op) {
 case "+": return l + r;
 case "*": return l * r;
 // ... other cases, errors, ...
 }
}
```



## 18.14

## Evaluating a Tree



An arithmetic tree!

```

class Node {
 Node left, right;
 int data; String op;
}

```

```

long postOrderEval(Node node) {
 if(node == null) throw new /* ... */;
 if(node.op == null) // no operation
 return node.data;
 long l = postOrderEval(node.left);
 long r = postOrderEval(node.right);
 switch(node.op) {
 case "+": return l + r;
 case "*": return l * r;
 // ... other cases, errors, ...
 }
}

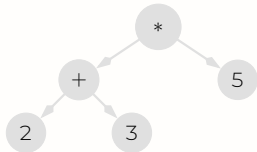
```

In practice one would realise the nodes differently. E.g. through inheritance.



## 18.15

## Evaluating a Tree



An arithmetic tree!

```
class Node {
 Node left, right;
 int data; String op;
}
```

```
long postOrderEval(Node node) {
 if(node == null) throw new /* ... */;
 if(node.op == null) // no operation
 return node.data;
 long l = postOrderEval(node.left);
 long r = postOrderEval(node.right);
 switch(node.op) {
 case "+": return l + r;
 case "*": return l * r;
 // ... other cases, errors, ...
 }
}
```

In practice one would realise the nodes differently. E.g. through inheritance.



## 18.16

## Evaluating a Tree



An arithmetic tree!

```
class Node {
 Node left, right;
 int data; String op;
}
```

```
long postOrderEval(Node node) {
 if(node == null) throw new /* ... */;
 if(node.op == null) // no operation
 return node.data;
 long l = postOrderEval(node.left);
 long r = postOrderEval(node.right);
 switch(node.op) {
 case "+": return l + r;
 case "*": return l * r;
 // ... other cases, errors, ...
 }
}
```

In practice one would realise the nodes differently. E.g. through inheritance.



In practice one would realise the nodes differently. E.g. through inheritance.



An arithmetic treat!

```
class Node {
 Node left, right;
 int data; String op;
}
```

```
long postOrderEval(Node node) {
 if(node == null) throw new /* ... */;
 if(node.op == null) // no operation
 return node.data;
 long l = postOrderEval(node.left);
 long r = postOrderEval(node.right);
 switch(node.op) {
 case "+": return l + r;
 case "*": return l * r;
 // ... other cases, errors, ...
 }
}
```

## 19.1

# The World of Recursive Problems



The Java Stack



Function Calls



Theory



**Examples**



Wrap-Up





## 19.2

# The World of Recursive Problems



The Java Stack



Function Calls



Theory



**Examples**



Wrap-Up



## 19.3

# The World of Recursive Problems

### 1. Recursive sorting



## 19.4

# The World of Recursive Problems

### 1. Recursive sorting



## 19.5

# The World of Recursive Problems

### 1. Recursive sorting



## 19.6

# The World of Recursive Problems

### 1. Recursive sorting



## 19.7

# The World of Recursive Problems

### 1. Recursive sorting



## 19.8

# The World of Recursive Problems

### 1. Recursive sorting



### 2. The towers of Hanoi



## 19.9

# The World of Recursive Problems

### 1. Recursive sorting



### 2. The towers of Hanoi





## 19.10

# The World of Recursive Problems

### 1. Recursive sorting



### 2. The towers of Hanoi



## 19.11

# The World of Recursive Problems

### 1. Recursive sorting



### 2. The towers of Hanoi



## 19.12

# The World of Recursive Problems

### 1. Recursive sorting



### 2. The towers of Hanoi



## 19.13

# The World of Recursive Problems

### 1. Recursive sorting



### 2. The towers of Hanoi



## 19.14

# The World of Recursive Problems

### 1. Recursive sorting



### 2. The towers of Hanoi



## 19.15

# The World of Recursive Problems

### 1. Recursive sorting



### 2. The towers of Hanoi



## 19.16

# The World of Recursive Problems

### 1. Recursive sorting



### 2. The towers of Hanoi



## 19.17

# The World of Recursive Problems

1. Recursive sorting



3. The N-queens problem

2. The towers of Hanoi





## 19.18

# The World of Recursive Problems

1. Recursive sorting



3. The N-queens problem



2. The towers of Hanoi



## 19.19

# The World of Recursive Problems

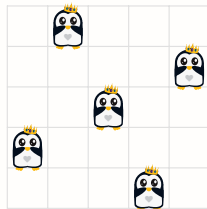
1. Recursive sorting



2. The towers of Hanoi



3. The N-queens problem



# The World of Recursive Problems

## 1. Recursive sorting

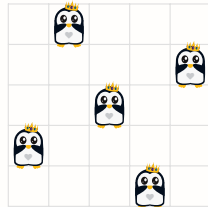


- Sudokus
- Pathfinding
- Hilbert-Kurven
- ...

## 2. The towers of Hanoi



## 3. The N-queens problem



## 19.21

# The World of Recursive Problems

### 1. Recursive sorting

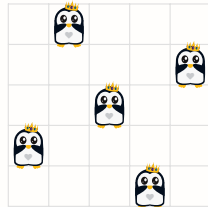


- Sudokus
- Pathfinding
- Hilbert-Kurven
- ...

### 2. The towers of Hanoi



### 3. The N-queens problem



Everything that can be split into (possibly explorative) single steps.



## 20.1

# Recursive Sorting

## Quicksort



The Java Stack



Function Calls



Theory



**Examples**



Wrap-Up



## 20.2

# Recursive Sorting

## Quicksort

```
void quicksort(int[] arr) {
}
```



## 20.3

# Recursive Sorting

## Quicksort

```
void quicksort(int[] arr) {
 quicksort(arr, 0, arr.length - 1);
}
```



## 20.4 Recursive Sorting

## 20.4 Recursive Sorting

## Quicksort

```
void quicksort(int[] arr) {
 quicksort(arr, 0, arr.length - 1);
}

void quicksort(int[] arr, int left, int right) {
}
}
```



## 20.5

# Recursive Sorting

## Quicksort

```
void quicksort(int[] arr) {
 quicksort(arr, 0, arr.length - 1);
}

void quicksort(int[] arr, int left, int right) {
 if(low >= right) return;

}
```

## 20.6

# Recursive Sorting

## Quicksort

```
void quicksort(int[] arr) {
 quicksort(arr, 0, arr.length - 1);
}

void quicksort(int[] arr, int left, int right) {
 if(low >= right) return;
 int partIndex = partition(arr, left, right);

}
```

```
void quicksort(int[] arr) {
 quicksort(arr, 0, arr.length - 1);
}

void quicksort(int[] arr, int left, int right) {
 if(low >= right) return;
 int partIndex = partition(arr, left, right);
 quicksort(arr, left, partIndex - 1);
 quicksort(arr, partIndex + 1, right);
}
```

## 20.8

# Recursive Sorting

## Quicksort

```
void quicksort(int[] arr) {
 quicksort(arr, 0, arr.length - 1);
}
```

```
void quicksort(int[] arr, int left, int right) {
 if(low >= right) return;
 int partIndex = partition(arr, left, right);
 quicksort(arr, left, partIndex - 1);
 quicksort(arr, partIndex + 1, right);
}
```

Picks a pivot and sorts everything smaller than the pivot to the left and everything greater or equal to the right.

## 21.1

# The Towers of Hanoi



## 21.2

# The Towers of Hanoi



A



B



C



The Java Stack



Function Calls



Theory



**Examples**



Wrap-Up



## 21.3

# The Towers of Hanoi



A



B



C

- Trivial for a single disk:  $A \rightarrow C$ .



## 21.4

# The Towers of Hanoi



- Trivial for a single disk:  $A \rightarrow C$ .
- Otherwise:
  - Move  $n - 1$  disks to the helper pole:  $A \rightarrow B$ .



## 21.5

# The Towers of Hanoi



- Trivial for a single disk:  $A \rightarrow C$ .
- Otherwise:
  - Move  $n - 1$  disks to the helper pole:  $A \rightarrow B$ .
  - Move the last (largest) disk from  $A \rightarrow C$ .

## 21.6

# The Towers of Hanoi



- Trivial for a single disk:  $A \rightarrow C$ .
- Otherwise:
  - Move  $n - 1$  disks to the helper pole:  $A \rightarrow B$ .
  - Move the last (largest) disk from  $A \rightarrow C$ .
  - Move  $n - 1$  disks from the helper pole to the target:  $B \rightarrow C$ .

## 21.7

# The Towers of Hanoi



- Trivial for a single disk:  $A \rightarrow C$ .
- Otherwise:
  - Move  $n - 1$  disks to the helper pole:  $A \rightarrow B$ .
  - Move the last (largest) disk from  $A \rightarrow C$ .
  - Move  $n - 1$  disks from the helper pole to the target:  $B \rightarrow C$ .

```
enum Pole {
 SOURCE,
 HELPER,
 TARGET
}
```



## 21.8

# The Towers of Hanoi



- Trivial for a single disk:  $A \rightarrow C$ .
- Otherwise:
  - Move  $n - 1$  disks to the helper pole:  $A \rightarrow B$ .
  - Move the last (largest) disk from  $A \rightarrow C$ .
  - Move  $n - 1$  disks from the helper pole to the target:  $B \rightarrow C$ .

```
enum Pole {
 SOURCE,
 HELPER,
 TARGET
}
```

```
void hanoi(int disks, Pole a, Pole b, Pole c) {
```

```
}
```



## 21.9

# The Towers of Hanoi



- Trivial for a single disk:  $A \rightarrow C$ .
- Otherwise:
  - Move  $n - 1$  disks to the helper pole:  $A \rightarrow B$ .
  - Move the last (largest) disk from  $A \rightarrow C$ .
  - Move  $n - 1$  disks from the helper pole to the target:  $B \rightarrow C$ .

```
enum Pole {
 SOURCE,
 HELPER,
 TARGET
}
```

```
void hanoi(int disks, Pole a, Pole b, Pole c) {
 if(disks == 1) { move(a, c); }
 else {

 }
}
```



## 21.10

# The Towers of Hanoi



- Trivial for a single disk:  $A \rightarrow C$ .
- Otherwise:
  - Move  $n - 1$  disks to the helper pole:  $A \rightarrow B$ .
  - Move the last (largest) disk from  $A \rightarrow C$ .
  - Move  $n - 1$  disks from the helper pole to the target:  $B \rightarrow C$ .

```
enum Pole {
 SOURCE,
 HELPER,
 TARGET
}
```

```
void hanoi(int disks, Pole a, Pole b, Pole c) {
 if(disks == 1) { move(a, c); }
 else {
 hanoi(disks - 1, a, c, b); // move all disks above to help pole
 }
}
```



## 21.11

# The Towers of Hanoi



A



B



C

- Trivial for a single disk:  $A \rightarrow C$ .
- Otherwise:
  - Move  $n - 1$  disks to the helper pole:  $A \rightarrow B$ .
  - Move the last (largest) disk from  $A \rightarrow C$ .
  - Move  $n - 1$  disks from the helper pole to the target:  $B \rightarrow C$ .

```
enum Pole {
 SOURCE,
 HELPER,
 TARGET
}
```

```
void hanoi(int disks, Pole a, Pole b, Pole c) {
 if(disks == 1) { move(a, c); }
 else {
 hanoi(disks - 1, a, c, b); // move all disks above to help pole
 move(a, c); // move last disk to target
 }
}
```



## 21.12

# The Towers of Hanoi



- Trivial for a single disk:  $A \rightarrow C$ .
- Otherwise:
  - Move  $n - 1$  disks to the helper pole:  $A \rightarrow B$ .
  - Move the last (largest) disk from  $A \rightarrow C$ .
  - Move  $n - 1$  disks from the helper pole to the target:  $B \rightarrow C$ .

```
enum Pole {
 SOURCE,
 HELPER,
 TARGET
}
```

```
void hanoi(int disks, Pole a, Pole b, Pole c) {
 if(disks == 1) { move(a, c); }
 else {
 hanoi(disks - 1, a, c, b); // move all disks above to help pole
 move(a, c); // move last disk to target
 hanoi(disks - 1, b, a, c); // move disks from help to target pole
 }
}
```





## 21.13

## The Towers of Hanoi



- Trivial for a single disk:  $A \rightarrow C$ .
- Otherwise:
  - Move  $n - 1$  disks to the helper pole:  $A \rightarrow B$ .
  - Move the last (largest) disk from  $A \rightarrow C$ .
  - Move  $n - 1$  disks from the helper pole to the target:  $B \rightarrow C$ .

```
enum Pole {
 SOURCE,
 HELPER,
 TARGET
}
```

```
void hanoi(int disks, Pole a, Pole b, Pole c) {
 if(disks == 1) { move(a, c); }
 else {
 hanoi(disks - 1, a, c, b); // move all disks above to help pole
 move(a, c); // move last disk to target
 hanoi(disks - 1, b, a, c); // move disks from help to target pole
 }
}
```

In the simplest case it just prints " $a \rightarrow c$ ".



## 22.1

# The N-Queens Problem



## 22.2

# The N-Queens Problem

```
boolean nQueens(boolean[][] board) {
 return nQueens(board, 0);
}
```



## 22.3

# The N-Queens Problem

```
boolean nQueens(boolean[][] board) {
 return nQueens(board, 0);
}
```

```
boolean nQueens(boolean[][] board, int row) {
```

## 22.4

# The N-Queens Problem

```
boolean nQueens(boolean[][] board) {
 return nQueens(board, 0);
}
```

```
boolean nQueens(boolean[][] board, int row) {
 if(row == board.length) return true;

 }
```

```
}
```



## 22.5

# The N-Queens Problem

```
boolean nQueens(boolean[][] board) {
 return nQueens(board, 0);
}
```

```
boolean nQueens(boolean[][] board, int row) {
 if(row == board.length) return true;
 for (int column = 0; column < board.length; column++) {

 }
}
```



## 22.6

# The N-Queens Problem

```
boolean nQueens(boolean[][] board) {
 return nQueens(board, 0);
}
```

```
boolean nQueens(boolean[][] board, int row) {
 if(row == board.length) return true;
 for (int column = 0; column < board.length; column++) {
 if(!queenCanBePlacedHere(board, column, row)) continue;
 }
}
```

## 22.7

# The N-Queens Problem

```
boolean nQueens(boolean[][] board) {
 return nQueens(board, 0);
}
```

```
boolean nQueens(boolean[][] board, int row) {
 if(row == board.length) return true;
 for (int column = 0; column < board.length; column++) {
 if(!queenCanBePlacedHere(board, column, row)) continue;
 board[row][column] = true; // Try to place queen here
 }
}
```



## 22.8

# The N-Queens Problem

```
boolean nQueens(boolean[][] board) {
 return nQueens(board, 0);
}
```

```
boolean nQueens(boolean[][] board, int row) {
 if(row == board.length) return true;
 for (int column = 0; column < board.length; column++) {
 if(!queenCanBePlacedHere(board, column, row)) continue;
 board[row][column] = true; // Try to place queen here
 if(nQueens(board, row + 1))
 return true;
 }
}
```

## 22.9

# The N-Queens Problem

```
boolean nQueens(boolean[][] board) {
 return nQueens(board, 0);
}
```

```
boolean nQueens(boolean[][] board, int row) {
 if(row == board.length) return true;
 for (int column = 0; column < board.length; column++) {
 if(!queenCanBePlacedHere(board, column, row)) continue;
 board[row][column] = true; // Try to place queen here
 if(nQueens(board, row + 1))
 return true;
 board[row][column] = false;
 }
}
```

## 22.10

# The N-Queens Problem

```
boolean nQueens(boolean[][] board) {
 return nQueens(board, 0);
}
```

```
boolean nQueens(boolean[][] board, int row) {
 if(row == board.length) return true;
 for (int column = 0; column < board.length; column++) {
 if(!queenCanBePlacedHere(board, column, row)) continue;
 board[row][column] = true; // Try to place queen here
 if(nQueens(board, row + 1))
 return true;
 board[row][column] = false;
 }
 return false;
}
```

## 22.11

# The N-Queens Problem

```
boolean nQueens(boolean[][] board) {
 return nQueens(board, 0);
}
```

```
boolean nQueens(boolean[][] board, int row) {
 if(row == board.length) return true;
 for (int column = 0; column < board.length; column++) {
 if(!queenCanBePlacedHere(board, column, row)) continue;
 board[row][column] = true; // Try to place queen here
 if(nQueens(board, row + 1))
 return true;
 board[row][column] = false;
 }
 return false;
}
```

How could  
queenCanBePlacedHere  
be implemented?



## 22.12

## The N-Queens Problem

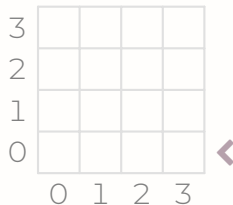
```
boolean nQueens(boolean[][] board) {
 return nQueens(board, 0);
}
```

```
• boolean nQueens(boolean[][] board, int row) {
 if(row == board.length) return true;
 for (int column = 0; column < board.length; column++) {
 if(!queenCanBePlacedHere(board, column, row)) continue;
 board[row][column] = true; // Try to place queen here
 if(nQueens(board, row + 1))
 return true;
 board[row][column] = false;
 }
 return false;
}
```

How could  
queenCanBePlacedHere  
be implemented?



q(board, 0)



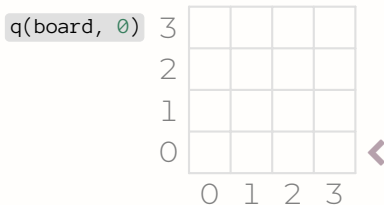
## 22.13

## The N-Queens Problem

```
boolean nQueens(boolean[][] board) {
 return nQueens(board, 0);
}
```

```
boolean nQueens(boolean[][] board, int row) {
 if(row == board.length) return true;
 for (int column = 0; column < board.length; column++) {
 if(!queenCanBePlacedHere(board, column, row)) continue;
 board[row][column] = true; // Try to place queen here
 if(nQueens(board, row + 1))
 return true;
 board[row][column] = false;
 }
 return false;
}
```

How could  
queenCanBePlacedHere  
be implemented?



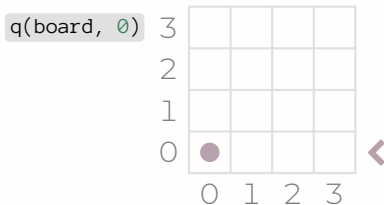
## 22.14

## The N-Queens Problem

```
boolean nQueens(boolean[][] board) {
 return nQueens(board, 0);
}
```

```
boolean nQueens(boolean[][] board, int row) {
 if(row == board.length) return true;
 • for (int column = 0; column < board.length; column++) {
 if(!queenCanBePlacedHere(board, column, row)) continue;
 board[row][column] = true; // Try to place queen here
 if(nQueens(board, row + 1))
 return true;
 board[row][column] = false;
 }
 return false;
}
```

How could  
queenCanBePlacedHere  
be implemented?



## 22.15

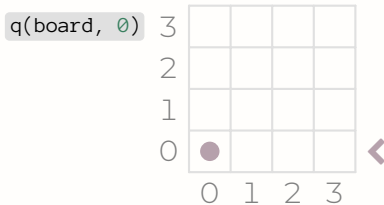
## The N-Queens Problem

```
boolean nQueens(boolean[][] board) {
 return nQueens(board, 0);
}
```

How could  
queenCanBePlacedHere  
be implemented?



```
boolean nQueens(boolean[][] board, int row) {
 if(row == board.length) return true;
 for (int column = 0; column < board.length; column++) {
 if(!queenCanBePlacedHere(board, column, row)) continue;
 board[row][column] = true; // Try to place queen here
 if(nQueens(board, row + 1))
 return true;
 board[row][column] = false;
 }
 return false;
}
```





## 22.16

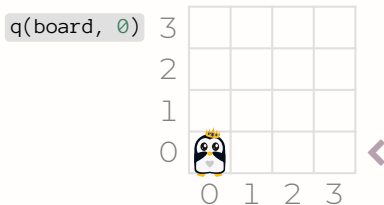
## The N-Queens Problem

```
boolean nQueens(boolean[][] board) {
 return nQueens(board, 0);
}
```

How could  
queenCanBePlacedHere  
be implemented?



```
boolean nQueens(boolean[][] board, int row) {
 if(row == board.length) return true;
 for (int column = 0; column < board.length; column++) {
 if(!queenCanBePlacedHere(board, column, row)) continue;
 board[row][column] = true; // Try to place queen here
 if(nQueens(board, row + 1))
 return true;
 board[row][column] = false;
 }
 return false;
}
```



## 22.17

## The N-Queens Problem

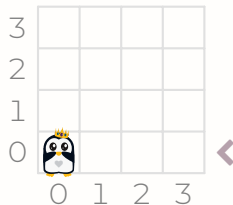
```
boolean nQueens(boolean[][] board) {
 return nQueens(board, 0);
}
```

```
boolean nQueens(boolean[][] board, int row) {
 if(row == board.length) return true;
 for (int column = 0; column < board.length; column++) {
 if(!queenCanBePlacedHere(board, column, row)) continue;
 board[row][column] = true; // Try to place queen here
 if(nQueens(board, row + 1))
 return true;
 board[row][column] = false;
 }
 return false;
}
```

How could  
queenCanBePlacedHere  
be implemented?



q(board, 0)



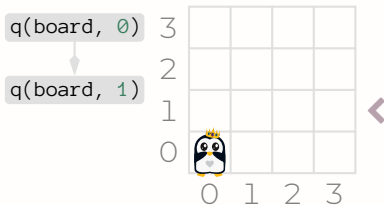
## 22.18

## The N-Queens Problem

```
boolean nQueens(boolean[][] board) {
 return nQueens(board, 0);
}
```

- ```
boolean nQueens(boolean[][] board, int row) {  
    if(row == board.length) return true;  
    for (int column = 0; column < board.length; column++) {  
        if(!queenCanBePlacedHere(board, column, row)) continue;  
        board[row][column] = true; // Try to place queen here  
        if(nQueens(board, row + 1))  
            return true;  
        board[row][column] = false;  
    }  
    return false;  
}
```

How could
queenCanBePlacedHere
be implemented?



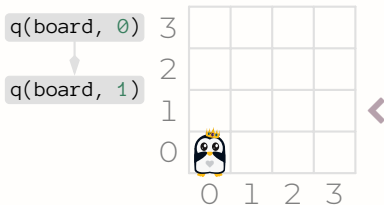
22.19

The N-Queens Problem

```
boolean nQueens(boolean[][] board) {
    return nQueens(board, 0);
}
```

```
boolean nQueens(boolean[][] board, int row) {
    if(row == board.length) return true;
    for (int column = 0; column < board.length; column++) {
        if(!queenCanBePlacedHere(board, column, row)) continue;
        board[row][column] = true; // Try to place queen here
        if(nQueens(board, row + 1))
            return true;
        board[row][column] = false;
    }
    return false;
}
```

How could
queenCanBePlacedHere
be implemented?



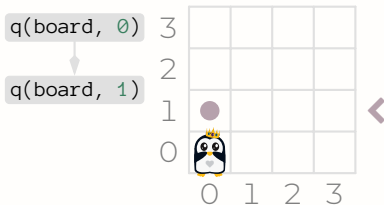
22.20

The N-Queens Problem

```
boolean nQueens(boolean[][] board) {
    return nQueens(board, 0);
}
```

```
boolean nQueens(boolean[][] board, int row) {
    if(row == board.length) return true;
    • for (int column = 0; column < board.length; column++) {
        if(!queenCanBePlacedHere(board, column, row)) continue;
        board[row][column] = true; // Try to place queen here
        if(nQueens(board, row + 1))
            return true;
        board[row][column] = false;
    }
    return false;
}
```

How could
queenCanBePlacedHere
be implemented?



22.21

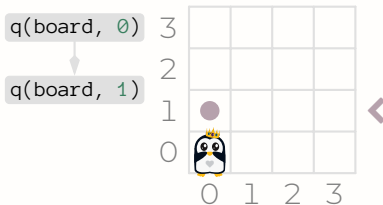
The N-Queens Problem

```
boolean nQueens(boolean[][] board) {  
    return nQueens(board, 0);  
}
```

How could
queenCanBePlacedHere
be implemented?



```
boolean nQueens(boolean[][] board, int row) {  
    if(row == board.length) return true;  
    for (int column = 0; column < board.length; column++) {  
        if(!queenCanBePlacedHere(board, column, row)) continue;  
        board[row][column] = true; // Try to place queen here  
        if(nQueens(board, row + 1))  
            return true;  
        board[row][column] = false;  
    }  
    return false;  
}
```



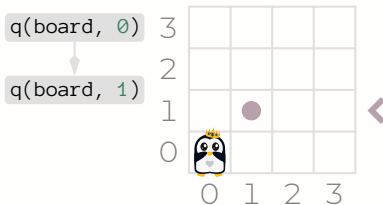
22.22

The N-Queens Problem

```
boolean nQueens(boolean[][] board) {  
    return nQueens(board, 0);  
}
```

```
boolean nQueens(boolean[][] board, int row) {  
    if(row == board.length) return true;  
    • for (int column = 0; column < board.length; column++) {  
        if(!queenCanBePlacedHere(board, column, row)) continue;  
        board[row][column] = true; // Try to place queen here  
        if(nQueens(board, row + 1))  
            return true;  
        board[row][column] = false;  
    }  
    return false;  
}
```

How could
queenCanBePlacedHere
be implemented?



22.23

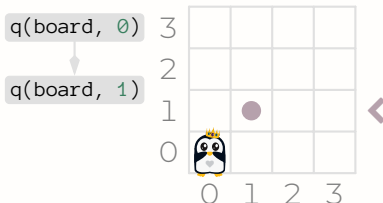
The N-Queens Problem

```
boolean nQueens(boolean[][] board) {
    return nQueens(board, 0);
}
```

How could
queenCanBePlacedHere
be implemented?



```
boolean nQueens(boolean[][] board, int row) {
    if(row == board.length) return true;
    for (int column = 0; column < board.length; column++) {
        if(!queenCanBePlacedHere(board, column, row)) continue;
        board[row][column] = true; // Try to place queen here
        if(nQueens(board, row + 1))
            return true;
        board[row][column] = false;
    }
    return false;
}
```



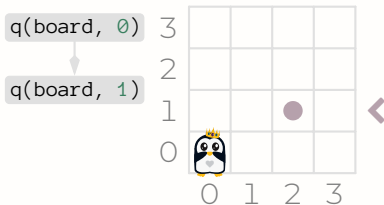
22.24

The N-Queens Problem

```
boolean nQueens(boolean[][] board) {
    return nQueens(board, 0);
}
```

```
boolean nQueens(boolean[][] board, int row) {
    if(row == board.length) return true;
    • for (int column = 0; column < board.length; column++) {
        if(!queenCanBePlacedHere(board, column, row)) continue;
        board[row][column] = true; // Try to place queen here
        if(nQueens(board, row + 1))
            return true;
        board[row][column] = false;
    }
    return false;
}
```

How could
queenCanBePlacedHere
be implemented?



22.25

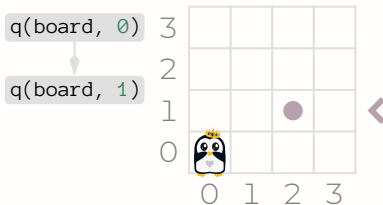
The N-Queens Problem

```
boolean nQueens(boolean[][] board) {
    return nQueens(board, 0);
}
```

How could
queenCanBePlacedHere
be implemented?



```
boolean nQueens(boolean[][] board, int row) {
    if(row == board.length) return true;
    for (int column = 0; column < board.length; column++) {
        if(!queenCanBePlacedHere(board, column, row)) continue;
        board[row][column] = true; // Try to place queen here
        if(nQueens(board, row + 1))
            return true;
        board[row][column] = false;
    }
    return false;
}
```



22.26

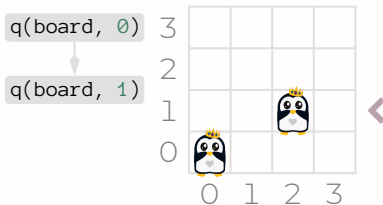
The N-Queens Problem

```
boolean nQueens(boolean[][] board) {
    return nQueens(board, 0);
}
```

How could
queenCanBePlacedHere
be implemented?



```
boolean nQueens(boolean[][] board, int row) {
    if(row == board.length) return true;
    for (int column = 0; column < board.length; column++) {
        if(!queenCanBePlacedHere(board, column, row)) continue;
        board[row][column] = true; // Try to place queen here
        if(nQueens(board, row + 1))
            return true;
        board[row][column] = false;
    }
    return false;
}
```



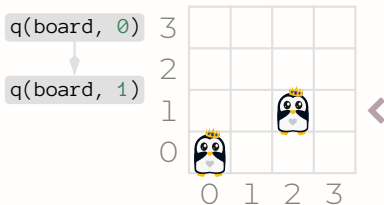
22.27

The N-Queens Problem

```
boolean nQueens(boolean[][] board) {
    return nQueens(board, 0);
}
```

```
boolean nQueens(boolean[][] board, int row) {
    if(row == board.length) return true;
    for (int column = 0; column < board.length; column++) {
        if(!queenCanBePlacedHere(board, column, row)) continue;
        board[row][column] = true; // Try to place queen here
        if(nQueens(board, row + 1))
            return true;
        board[row][column] = false;
    }
    return false;
}
```

How could
queenCanBePlacedHere
be implemented?



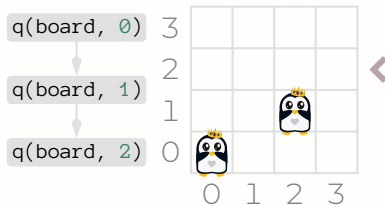
22.28

The N-Queens Problem

```
boolean nQueens(boolean[][] board) {  
    return nQueens(board, 0);  
}
```

- ```
boolean nQueens(boolean[][] board, int row) {
 if(row == board.length) return true;
 for (int column = 0; column < board.length; column++) {
 if(!queenCanBePlacedHere(board, column, row)) continue;
 board[row][column] = true; // Try to place queen here
 if(nQueens(board, row + 1))
 return true;
 board[row][column] = false;
 }
 return false;
}
```

How could  
queenCanBePlacedHere  
be implemented?



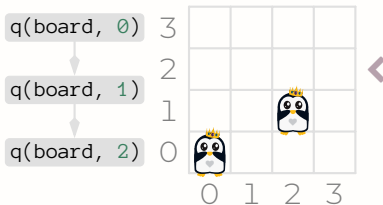
## 22.29

## The N-Queens Problem

```
boolean nQueens(boolean[][] board) {
 return nQueens(board, 0);
}
```

```
boolean nQueens(boolean[][] board, int row) {
 if(row == board.length) return true;
 for (int column = 0; column < board.length; column++) {
 if(!queenCanBePlacedHere(board, column, row)) continue;
 board[row][column] = true; // Try to place queen here
 if(nQueens(board, row + 1))
 return true;
 board[row][column] = false;
 }
 return false;
}
```

How could  
queenCanBePlacedHere  
be implemented?



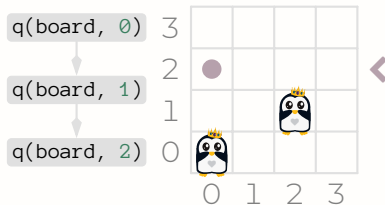
## 22.30

## The N-Queens Problem

```
boolean nQueens(boolean[][] board) {
 return nQueens(board, 0);
}
```

```
boolean nQueens(boolean[][] board, int row) {
 if(row == board.length) return true;
 for (int column = 0; column < board.length; column++) {
 if(!queenCanBePlacedHere(board, column, row)) continue;
 board[row][column] = true; // Try to place queen here
 if(nQueens(board, row + 1))
 return true;
 board[row][column] = false;
 }
 return false;
}
```

How could  
queenCanBePlacedHere  
be implemented?



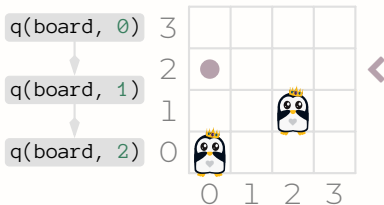
## 22.31

## The N-Queens Problem

```
boolean nQueens(boolean[][] board) {
 return nQueens(board, 0);
}
```

```
boolean nQueens(boolean[][] board, int row) {
 if(row == board.length) return true;
 for (int column = 0; column < board.length; column++) {
 if(!queenCanBePlacedHere(board, column, row)) continue;
 board[row][column] = true; // Try to place queen here
 if(nQueens(board, row + 1))
 return true;
 board[row][column] = false;
 }
 return false;
}
```

How could  
queenCanBePlacedHere  
be implemented?





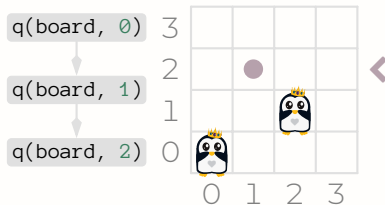
## 22.32

## The N-Queens Problem

```
boolean nQueens(boolean[][] board) {
 return nQueens(board, 0);
}
```

```
boolean nQueens(boolean[][] board, int row) {
 if(row == board.length) return true;
 • for (int column = 0; column < board.length; column++) {
 if(!queenCanBePlacedHere(board, column, row)) continue;
 board[row][column] = true; // Try to place queen here
 if(nQueens(board, row + 1))
 return true;
 board[row][column] = false;
 }
 return false;
}
```

How could  
queenCanBePlacedHere  
be implemented?



## 22.33

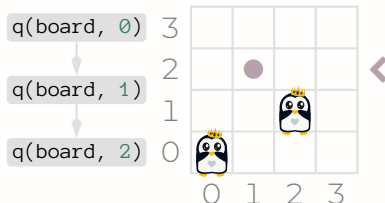
## The N-Queens Problem

```
boolean nQueens(boolean[][] board) {
 return nQueens(board, 0);
}
```

How could  
queenCanBePlacedHere  
be implemented?



```
boolean nQueens(boolean[][] board, int row) {
 if(row == board.length) return true;
 for (int column = 0; column < board.length; column++) {
 if(!queenCanBePlacedHere(board, column, row)) continue;
 board[row][column] = true; // Try to place queen here
 if(nQueens(board, row + 1))
 return true;
 board[row][column] = false;
 }
 return false;
}
```



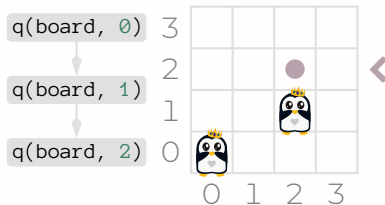
## 22.34

## The N-Queens Problem

```
boolean nQueens(boolean[][] board) {
 return nQueens(board, 0);
}
```

```
boolean nQueens(boolean[][] board, int row) {
 if(row == board.length) return true;
 • for (int column = 0; column < board.length; column++) {
 if(!queenCanBePlacedHere(board, column, row)) continue;
 board[row][column] = true; // Try to place queen here
 if(nQueens(board, row + 1))
 return true;
 board[row][column] = false;
 }
 return false;
}
```

How could  
queenCanBePlacedHere  
be implemented?



## 22.35

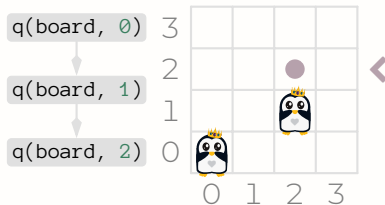
## The N-Queens Problem

```
boolean nQueens(boolean[][] board) {
 return nQueens(board, 0);
}
```

How could  
queenCanBePlacedHere  
be implemented?



```
boolean nQueens(boolean[][] board, int row) {
 if(row == board.length) return true;
 for (int column = 0; column < board.length; column++) {
 if(!queenCanBePlacedHere(board, column, row)) continue;
 board[row][column] = true; // Try to place queen here
 if(nQueens(board, row + 1))
 return true;
 board[row][column] = false;
 }
 return false;
}
```



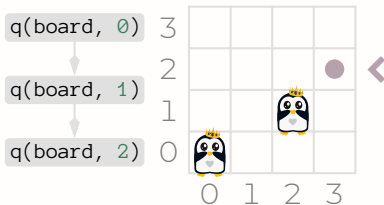
## 22.36

## The N-Queens Problem

```
boolean nQueens(boolean[][] board) {
 return nQueens(board, 0);
}
```

```
boolean nQueens(boolean[][] board, int row) {
 if(row == board.length) return true;
 for (int column = 0; column < board.length; column++) {
 if(!queenCanBePlacedHere(board, column, row)) continue;
 board[row][column] = true; // Try to place queen here
 if(nQueens(board, row + 1))
 return true;
 board[row][column] = false;
 }
 return false;
}
```

How could  
queenCanBePlacedHere  
be implemented?



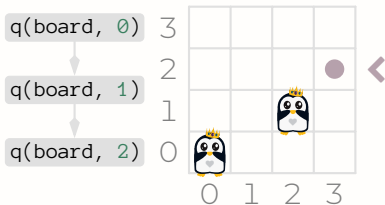
## 22.37

## The N-Queens Problem

```
boolean nQueens(boolean[][] board) {
 return nQueens(board, 0);
}
```

```
boolean nQueens(boolean[][] board, int row) {
 if(row == board.length) return true;
 for (int column = 0; column < board.length; column++) {
 if(!queenCanBePlacedHere(board, column, row)) continue;
 board[row][column] = true; // Try to place queen here
 if(nQueens(board, row + 1))
 return true;
 board[row][column] = false;
 }
 return false;
}
```

How could  
queenCanBePlacedHere  
be implemented?



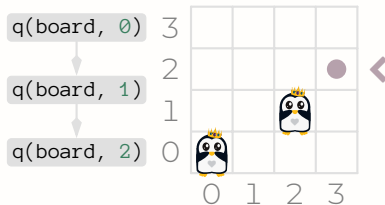
## 22.38

## The N-Queens Problem

```
boolean nQueens(boolean[][] board) {
 return nQueens(board, 0);
}
```

```
boolean nQueens(boolean[][] board, int row) {
 if(row == board.length) return true;
 for (int column = 0; column < board.length; column++) {
 if(!queenCanBePlacedHere(board, column, row)) continue;
 board[row][column] = true; // Try to place queen here
 if(nQueens(board, row + 1))
 return true;
 board[row][column] = false;
 }
 return false;
}
```

How could  
queenCanBePlacedHere  
be implemented?



## 22.39

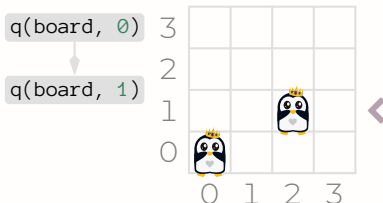
## The N-Queens Problem

```
boolean nQueens(boolean[][] board) {
 return nQueens(board, 0);
}
```

How could  
queenCanBePlacedHere  
be implemented?



```
boolean nQueens(boolean[][] board, int row) {
 if(row == board.length) return true;
 for (int column = 0; column < board.length; column++) {
 if(!queenCanBePlacedHere(board, column, row)) continue;
 board[row][column] = true; // Try to place queen here
 if(nQueens(board, row + 1))
 return true;
 board[row][column] = false;
 }
 return false;
}
```





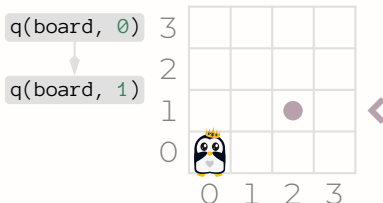
## 22.40

## The N-Queens Problem

```
boolean nQueens(boolean[][] board) {
 return nQueens(board, 0);
}
```

```
boolean nQueens(boolean[][] board, int row) {
 if(row == board.length) return true;
 for (int column = 0; column < board.length; column++) {
 if(!queenCanBePlacedHere(board, column, row)) continue;
 board[row][column] = true; // Try to place queen here
 if(nQueens(board, row + 1))
 return true;
 board[row][column] = false;
 }
 return false;
}
```

How could  
queenCanBePlacedHere  
be implemented?



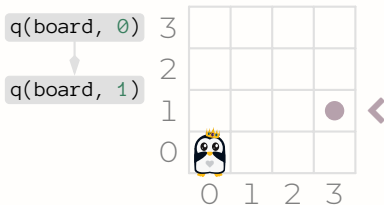
## 22.41

## The N-Queens Problem

```
boolean nQueens(boolean[][] board) {
 return nQueens(board, 0);
}
```

```
boolean nQueens(boolean[][] board, int row) {
 if(row == board.length) return true;
 • for (int column = 0; column < board.length; column++) {
 if(!queenCanBePlacedHere(board, column, row)) continue;
 board[row][column] = true; // Try to place queen here
 if(nQueens(board, row + 1))
 return true;
 board[row][column] = false;
 }
 return false;
}
```

How could  
queenCanBePlacedHere  
be implemented?



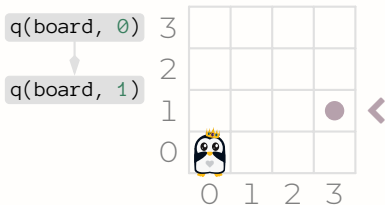
## 22.42

## The N-Queens Problem

```
boolean nQueens(boolean[][] board) {
 return nQueens(board, 0);
}
```

```
boolean nQueens(boolean[][] board, int row) {
 if(row == board.length) return true;
 for (int column = 0; column < board.length; column++) {
 if(!queenCanBePlacedHere(board, column, row)) continue;
 board[row][column] = true; // Try to place queen here
 if(nQueens(board, row + 1))
 return true;
 board[row][column] = false;
 }
 return false;
}
```

How could  
queenCanBePlacedHere  
be implemented?



## 22.43

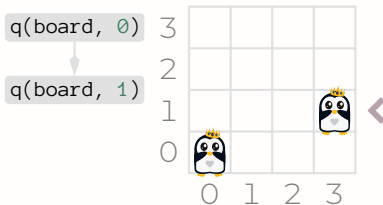
## The N-Queens Problem

```
boolean nQueens(boolean[][] board) {
 return nQueens(board, 0);
}
```

How could  
queenCanBePlacedHere  
be implemented?



```
boolean nQueens(boolean[][] board, int row) {
 if(row == board.length) return true;
 for (int column = 0; column < board.length; column++) {
 if(!queenCanBePlacedHere(board, column, row)) continue;
 board[row][column] = true; // Try to place queen here
 if(nQueens(board, row + 1))
 return true;
 board[row][column] = false;
 }
 return false;
}
```



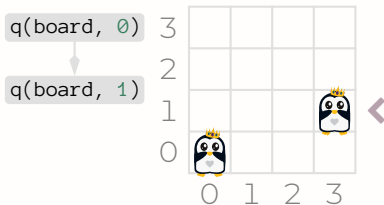
## 22.44

## The N-Queens Problem

```
boolean nQueens(boolean[][] board) {
 return nQueens(board, 0);
}
```

```
boolean nQueens(boolean[][] board, int row) {
 if(row == board.length) return true;
 for (int column = 0; column < board.length; column++) {
 if(!queenCanBePlacedHere(board, column, row)) continue;
 board[row][column] = true; // Try to place queen here
 if(nQueens(board, row + 1))
 return true;
 board[row][column] = false;
 }
 return false;
}
```

How could  
queenCanBePlacedHere  
be implemented?



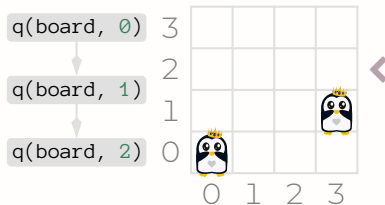
## 22.45

## The N-Queens Problem

```
boolean nQueens(boolean[][] board) {
 return nQueens(board, 0);
}
```

```
• boolean nQueens(boolean[][] board, int row) {
 if(row == board.length) return true;
 for (int column = 0; column < board.length; column++) {
 if(!queenCanBePlacedHere(board, column, row)) continue;
 board[row][column] = true; // Try to place queen here
 if(nQueens(board, row + 1))
 return true;
 board[row][column] = false;
 }
 return false;
}
```

How could  
queenCanBePlacedHere  
be implemented?



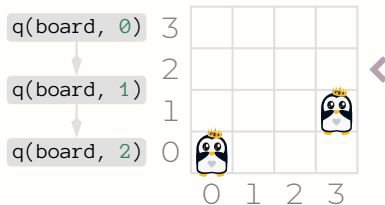
## 22.46

## The N-Queens Problem

```
boolean nQueens(boolean[][] board) {
 return nQueens(board, 0);
}
```

```
boolean nQueens(boolean[][] board, int row) {
 if(row == board.length) return true;
 for (int column = 0; column < board.length; column++) {
 if(!queenCanBePlacedHere(board, column, row)) continue;
 board[row][column] = true; // Try to place queen here
 if(nQueens(board, row + 1))
 return true;
 board[row][column] = false;
 }
 return false;
}
```

How could  
queenCanBePlacedHere  
be implemented?



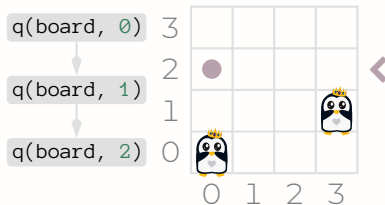
## 22.47

## The N-Queens Problem

```
boolean nQueens(boolean[][] board) {
 return nQueens(board, 0);
}
```

```
boolean nQueens(boolean[][] board, int row) {
 if(row == board.length) return true;
 • for (int column = 0; column < board.length; column++) {
 if(!queenCanBePlacedHere(board, column, row)) continue;
 board[row][column] = true; // Try to place queen here
 if(nQueens(board, row + 1))
 return true;
 board[row][column] = false;
 }
 return false;
}
```

How could  
queenCanBePlacedHere  
be implemented?





## 22.48

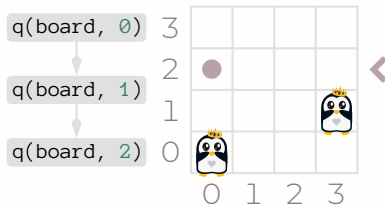
## The N-Queens Problem

```
boolean nQueens(boolean[][] board) {
 return nQueens(board, 0);
}
```

How could  
queenCanBePlacedHere  
be implemented?



```
boolean nQueens(boolean[][] board, int row) {
 if(row == board.length) return true;
 for (int column = 0; column < board.length; column++) {
 if(!queenCanBePlacedHere(board, column, row)) continue;
 board[row][column] = true; // Try to place queen here
 if(nQueens(board, row + 1))
 return true;
 board[row][column] = false;
 }
 return false;
}
```



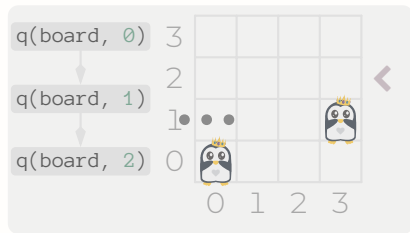
## 22.49

## The N-Queens Problem

```
boolean nQueens(boolean[][] board) {
 return nQueens(board, 0);
}
```

```
boolean nQueens(boolean[][] board, int row) {
 if(row == board.length) return true;
 for (int column = 0; column < board.length; column++) {
 if(!queenCanBePlacedHere(board, column, row)) continue;
 board[row][column] = true; // Try to place queen here
 if(nQueens(board, row + 1))
 return true;
 board[row][column] = false;
 }
 return false;
}
```

How could  
queenCanBePlacedHere  
be implemented?



## 22.50

# The N-Queens Problem

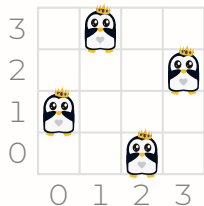
```
boolean nQueens(boolean[][] board) {
 return nQueens(board, 0);
}
```

How could  
queenCanBePlacedHere  
be implemented?



```
boolean nQueens(boolean[][] board, int row) {
 if(row == board.length) return true;
 for (int column = 0; column < board.length; column++) {
 if(!queenCanBePlacedHere(board, column, row)) continue;
 board[row][column] = true; // Try to place queen here
 if(nQueens(board, row + 1))
 return true;
 board[row][column] = false;
 }
 return false;
}
```

q(board, 0)



## 23.1

# The Obligatory “By the Way...”

[1]: *Proper Tail Recursion and Space Efficiency*  
Clinger, 1998



## 23.2

# The Obligatory “By the Way...”

- Many variants:

[1]: *Proper Tail Recursion and Space Efficiency*  
Clinger, 1998



## 23.3

# The Obligatory “By the Way...”

- Many variants:
  - Tail recursion

[1]: *Proper Tail Recursion and Space Efficiency*  
Clinger, 1998

## 23.4

# The Obligatory “By the Way...”

- Many variants:
  - Tail recursion
  - Primitive recursive functions

[1]: *Proper Tail Recursion and Space Efficiency*  
Clinger, 1998

## 23.5

# The Obligatory “By the Way...”

- Many variants:
  - Tail recursion
  - Primitive recursive functions
- Stack frames are more exciting and hold even more!

[1]: *Proper Tail Recursion and Space Efficiency*  
Clinger, 1998



## 23.6

# The Obligatory “By the Way...”

- Many variants:
  - Tail recursion
  - Primitive recursive functions
- Stack frames are more exciting and hold even more!
- The huge topic of compiler construction offers more:

[1]: *Proper Tail Recursion and Space Efficiency*  
Clinger, 1998

## 23.7

# The Obligatory “By the Way...”

- Many variants:
  - Tail recursion
  - Primitive recursive functions
- Stack frames are more exciting and hold even more!
- The huge topic of compiler construction offers more:
  - Optimizations & transformations

[1]: *Proper Tail Recursion and Space Efficiency*  
Clinger, 1998

## 23.8

# The Obligatory “By the Way...”

- Many variants:
  - Tail recursion
  - Primitive recursive functions
- Stack frames are more exciting and hold even more!
- The huge topic of compiler construction offers more:
  - Optimizations & transformations
  - Recursive Descent Parser

[1]: *Proper Tail Recursion and Space Efficiency*  
Clinger, 1998

## 23.9

# The Obligatory “By the Way...”

- Many variants:
  - Tail recursion
  - Primitive recursive functions
- Stack frames are more exciting and hold even more!
- The huge topic of compiler construction offers more:
  - Optimizations & transformations
  - Recursive Descent Parser
- Dynamic programming!

[1]: *Proper Tail Recursion and Space Efficiency*  
Clinger, 1998

## 23.10

# The Obligatory “By the Way...”

- Many variants:
  - Tail recursion
  - Primitive recursive functions
- Stack frames are more exciting and hold even more!
- The huge topic of compiler construction offers more:
  - Optimizations & transformations
  - Recursive Descent Parser
- Dynamic programming!

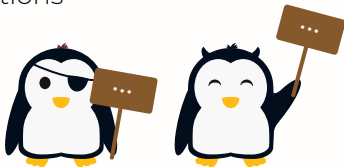


[1]: *Proper Tail Recursion and Space Efficiency*  
Clinger, 1998

## 23.11

# The Obligatory “By the Way...”

- Many variants:
  - Tail recursion
  - Primitive recursive functions
- Stack frames are more exciting and hold even more!
- The huge topic of compiler construction offers more:
  - Optimizations & transformations
  - Recursive Descent Parser
- Dynamic programming!



[1]: *Proper Tail Recursion and Space Efficiency*  
Clinger, 1998

## 23.12

# The Obligatory “By the Way...”

- Many variants:
  - Tail recursion
  - Primitive recursive functions
- Stack frames are more exciting and hold even more!
- The huge topic of compiler construction offers more:
  - Optimizations & transformations
  - Recursive Descent Parser
- Dynamic programming!



[1]: *Proper Tail Recursion and Space Efficiency*  
Clinger, 1998

## 24.1



The Java Stack



Function Calls



Theory



Examples

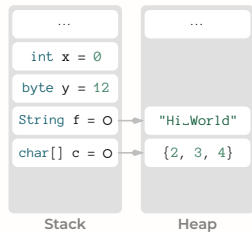


**Wrap-Up**





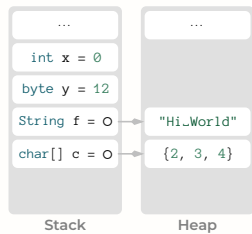
## 24.2



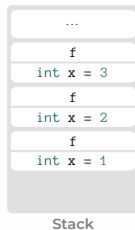
Memory  
management



## 24.3



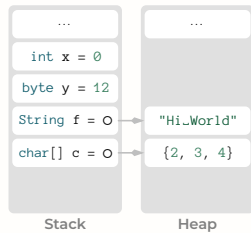
Memory  
management



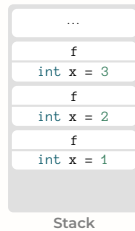
Stack Frames



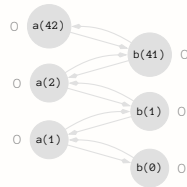
## 24.4



Memory  
management



Stack Frames



Recursion types

## 24.5



The Java Stack



Function Calls



Theory



Examples



**Wrap-Up**



## 24.6

$$f(n) = \begin{cases} 1, & n \leq 1 \\ n \cdot f(n-1), & n > 1 \end{cases}$$

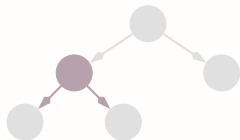
Mathematical



## 24.7

$$f(n) = \begin{cases} 1, & n \leq 1 \\ n \cdot f(n-1), & n > 1 \end{cases}$$

Mathematical

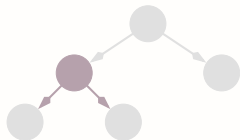


Data structures

## 24.8

$$f(n) = \begin{cases} 1, & n \leq 1 \\ n \cdot f(n-1), & n > 1 \end{cases}$$

Mathematical



Data structures



Divide- & Conquer



- [1] William D. Clinger. “Proper Tail Recursion and Space Efficiency”. 1998
- [2] Loïc Colson. “About primitive recursive algorithms”. 1991
- [3] *Java Debug Interface – StackFrame (JDK 11)*. 2021
- [4] *Lesson: Generics*. 2021
- [5] Tim Lindholm et al. *The Java® Virtual Machine Specification [Heap & Stack]*. 2020
- [6] Yanhong A. Liu and Scott D. Stoller. “From Recursion to Iteration: What Are the Optimizations?” Nov. 1999
- [7] Manuel Rubio-Sánchez and Isidoro Hernán-Losada. “Exploring Recursion with Fibonacci Numbers”. 2007
- [8] Florian Sihler. *L<sup>A</sup>T<sub>E</sub>X-Package, tikzpingus*. 2021
- [9] Yngve Sundblad. “The Ackermann function. a theoretical, computational, and formula manipulative study”. 1971



**Florian Sihler**

Ulm, September 21, 2026